points which affect the rate of change of temperature. Given a sinusoidal initial condition and zero boundary conditions, there should therefore not be differing rates of change of temperature at points equidistant from the midpoint and the nearest boundary. You managed to explain your observations across a large range of initial and boundary conditions; your general formulation of the problem allowed you to very cleanly vary between these. The bonus was done quite well.

Grade: C

7 Group 26

A decent report, generally covering the important parts of the analysis but prone to errors in some of the implementation and the resulting discussion. Presentation of the methodology employed, other than comments in the code, was lacking throughout.

7.1 Q1

You're missing any presentation of the methods used, which would have been particularly useful for Ralston's method since this was not covered in the course, although you did comment on the code itself. There is a fundamental flaw in your Midpoint function: *func* is defined with arguments x and y but they don't appear in its definition. As a result, the call to *func* evaluates it at (xt, yt) no matter what arguments are given, and so your function is in fact implementing the Euler method. Your parameters for Ralston's method are not the ones used as standard, although they do satisfy the requirements covered in the lectures.

The analysis was good but quite sparse - you could have put a bit more thought into the behaviour of the circuit. There was a key misconception that periodic attenuation was observed; it was the transients which were responsible for the observation of different system responses in the short- and long-term. The legibility of many of the figures needed improvement; the choice of horizontal scaling meant that it was difficult to pick out key aspects of the results demonstrated. The near-solid blocks demonstrating overall trends over long time periods in particular added little.

7.2 Q2

Re-use of the variable N meant that you only ran each numerical integration technique for 16 time steps on each loop. This meant that you were covering different proportions of the transient response each time, and so the comparison was not a representative one. This should have been apparent from the very little change with step size in the maximum absolute error values for the Midpoint and Ralston's methods. Additionally, none of your plots demonstrated that the error was of $\mathcal{O}(h^2)$ which you should have expected; in fact you incorrectly stated that you expected an exponential relationship, despite plotting on logarithmic axes.

7.3 Q3

Again, it would have been nice to have had some explanation of the code used in the main body of the text, but the comments in the code itself were good. The analysis was very good, considering the relative impedance of the capacitor and inductor as the frequency was varied. You mentioned that 110 Hz is very close to the resonant frequency, but you could have considered performing some calculations to find this exactly, as well as finding other filter properties of the circuit such as its bandwidth.

7.4 Q4

Once again, it would have been nice to have had some explanation of the code used in the main body of the text, but the comments in the code itself were good once again. Your analysis is very good, particularly drawing on the physical interpretation of the boundary conditions leading to the formation of heat sources or heat sinks. Your extension of the discussion to 3D objects was confusing - we are modelling the rod in 1D by assuming that it is thin and insulated along its length. Under these assumptions, the central difference offers a good approximation to each partial derivative, so we can approximately assume that it is only the temperatures of adjacent

EE2-08C Numerical analysis coursework
Group 26

Module Ref: EE2-08C
Submitted to: Dr. Daniel Nucinkis

Date: 15/03/17

Q1) 44
Q2) 8
Q3) 22
Q4) 18
57
58

Mikkel Jordahn CID: 00924570 mj2214@ic.ac.uk
George Punter CID: 00924627 gsp14@ic.ac.uk
Shreyus Bagga CID: 01070439 smb215@ic.ac.uk
Louis Kueh CID:01078797 lhk115@ic.ac.uk
Kaichun Hu CID:01055679 kh1615@ic.ac.uk

Table of Contents

1. RL CIRCUIT 3
1.1 STEP SIGNAL 3
1.2 IMPULSE SIGNAL AND DECAY 6
1.3 SINE WAVE AT VARIABLE FREQUENCIES 10
1.4 SQUARE WAVE WITH VARIABLE FREQUENCY 13

2. ERROR CALCULATION 19

3. RLC CIRCUIT 26
3.1 STEP FUNCTION INPUT 26
3.2 DECAYING STEP FUNCTION INPUT 27
3.3 5HZ SQUARE WAVE FUNCTION INPUT 28
3.4 110HZ SQUARE WAVE FUNCTION INPUT 29
3.5 500HZ SQUARE WAVE FUNCTION INPUT 30
3.6 5HZ SINE WAVE INPUT 31
3.7 110HZ SINE WAVE INPUT 32
3.8 500HZ SINE WAVE INPUT 33

4. FINITE DIFFERENCES FOR PDE 34
4.1 TENT FUNCTION, ... 35
4.2 SINE FUNCTION 36
4.3 ABS SIN FUNCTION 37
4.4 SINC FUNCTION – INITIAL CONDITION NOT EQUAL TO BOUNDARY CONDITION 38
4.6 SINC FUNCTION CONSTANT BOUNDARY CONDITION OF 1 40
4.6 SINC FUNCTION VARYING BOUNDARY CONDITION 41
4.7 HYPERBOLIC TAN FUNCTION ZERO BOUNDARY CONDITION 42
4.7 HYPERBOLIC TAN FUNCTION CONSTANT RIGHT BOUNDARY CONDITION 43

APPENDIX 45

1. RL Circuit

1 RL circuit

An high-pass filter takes an input signal V_{in} (for instance, an audio signal) and let only the high-frequency components pass. Usually, high-pass filters are made with capacitors instead of inductors due to the fact that inductors can be more easily manufactured and are generally physically smaller. However, the example of RL circuit we are going to consider is still very important. In fact, RL circuits are usually used in the modeling of real physical systems such as DC motors.

Consider the RL circuit as depicted in Figure 1. For the circuit, we can write the following equations:

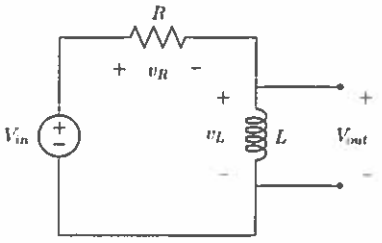


Figure 1: RL circuit

$$\begin{aligned} v_L(t) + v_R(t) &= V_{in}(t) \\ L\frac{d}{dt}i_L(t) + R\,i_L(t) &= V_{in}(t)\,. \end{aligned}$$

The state is $i_L(t)$. The input is $V_{in}(t)$. The output is $V_{out} = V_{in}(t) - Ri_L(t)$. Assume that the initial current through the inductor at time $t = 0$ is $i_L(0) = 0\,\mathrm{A}$. We want to model a DC motor with inertia $250\,\mathrm{pNm/s^2}$ and maximum torque gain $50\,\mathrm{mN\,m\,A^{-1}}$. The corresponding values for the inductance and resistance are:

$$\begin{aligned} R &= 0.5\,\Omega \\ L &= 1.5\,\mathrm{mH}\,. \end{aligned}$$

Figure 1

To obtain the required equation for the algorithm, the ODE was rearranged to the form seen in figure 2.

$$\frac{di_L(t)}{dt} = \frac{V_s - Ri_L(t)}{L}$$

Figure 2

In order to obtain the output voltage, the formula seen in figure 3 was used.

$$Vout = Vin - i(t)R$$

Figure 3

1.1 Step Signal

The first input was a step signal, with amplitude 5.5V. This was simulated by writing a step function script that implemented the following:

- $V = 5.5$ for $t > 0$
- $V = 0$ for $t < 0$

The resulting outputs can be seen for each method in the next section.

Heun's method:

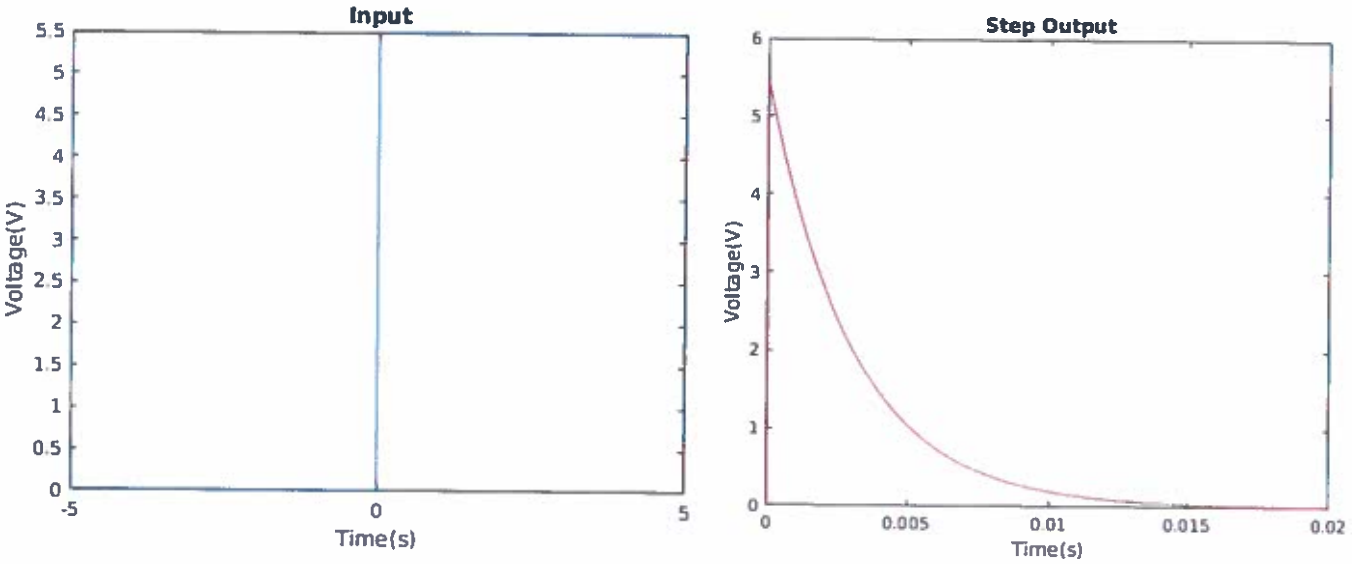


Figure 4

Ralston's method:

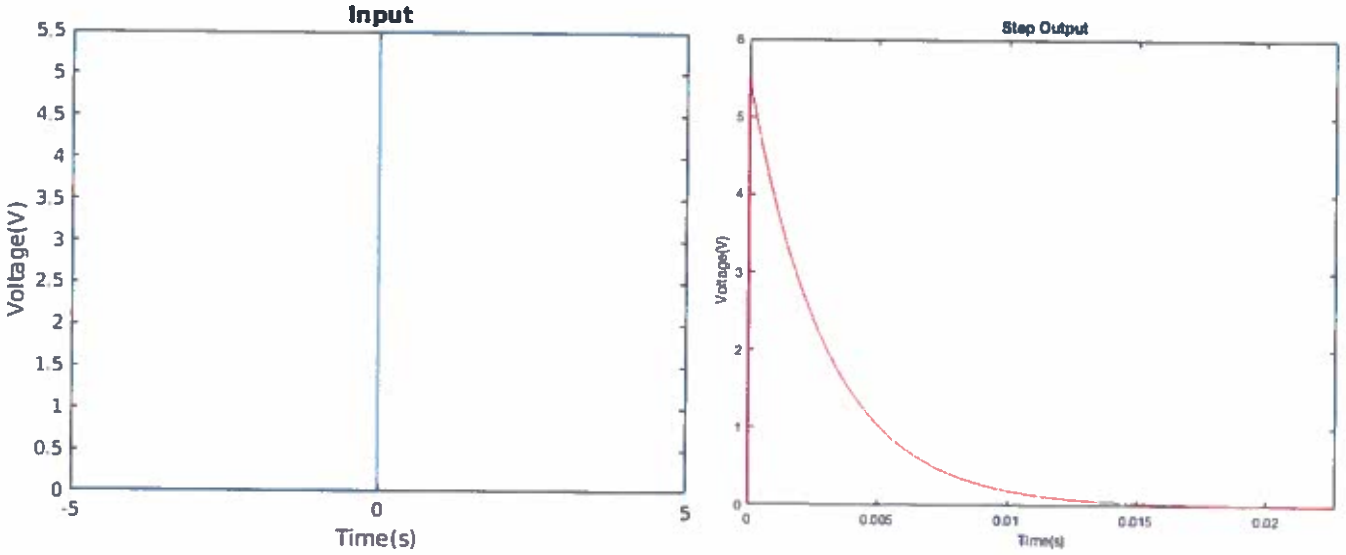


Figure 5

Midpoint method:

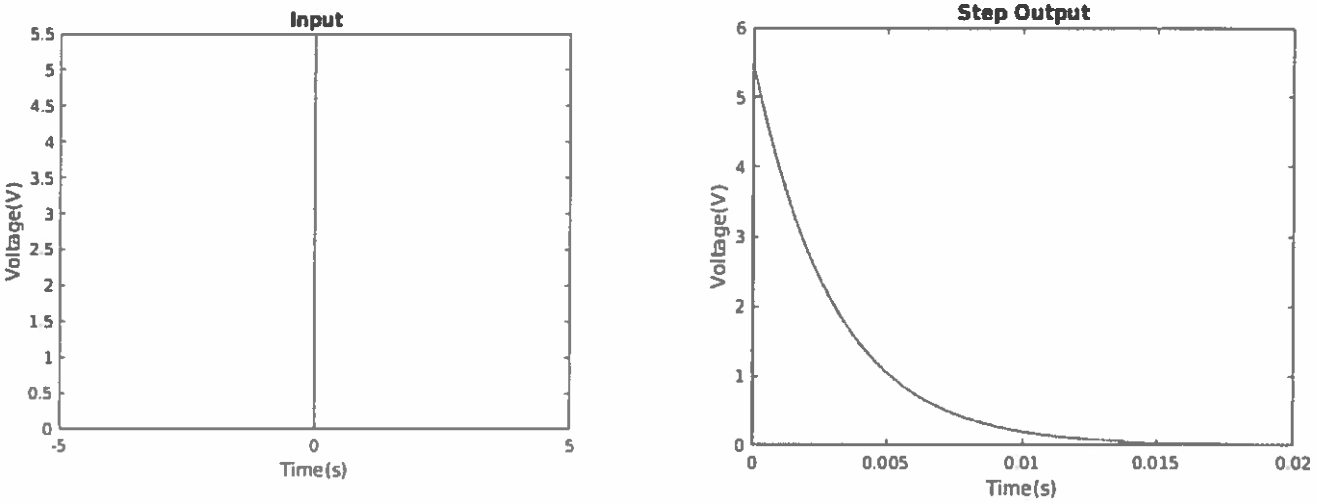


Figure 6

The step function has a voltage of $5.5V$ after $t > 0$, and 0 for $t < 0$ as expected. This is represented in the output graph by the sudden jump at 0 to $5.5V$, after which the voltage decays exponentially due to the RL circuit. In the circuit, the rate of decay will depend on the ratios of inductance to resistance. This is represented accurately in the output.

$$T = \frac{L}{R}, \qquad for\ RL\ Circuits$$

Figure 7

Since the equation for the transient curves for an RL series circuit is:

$$Vout = V\left(e^{-\frac{Rt}{L}}\right)$$

The above is true for all RL circuits but in this case the $R = 0.5$ and $L = 1.5\ mH$, giving a time constant of 0.003. The maximum steady state is roughly equivalent to 5 time constants, giving a value of $0.003 * 5 = 0.015$, which matches the output graph.

√ OK

1.2 Impulse signal and Decay

The next inputs are the impulsive signal and the decay signal, which can be seen in figure 8.

$$V_{in} = \bar{V}_{in} \exp\left\{-\frac{t^2}{\tau}\right\} \qquad V_{in} = \bar{V}_{in} \exp\left\{-\frac{t}{\tau}\right\}$$

with $\bar{V}_{in} = 3.5\,\mathrm{V}$ and $\tau = 160\,(\mu\mathrm{s})^2$, resp. $\tau = 160\,\mu\mathrm{s}$.

Figure 8

The results can be seen in the following section.

Heun's method:

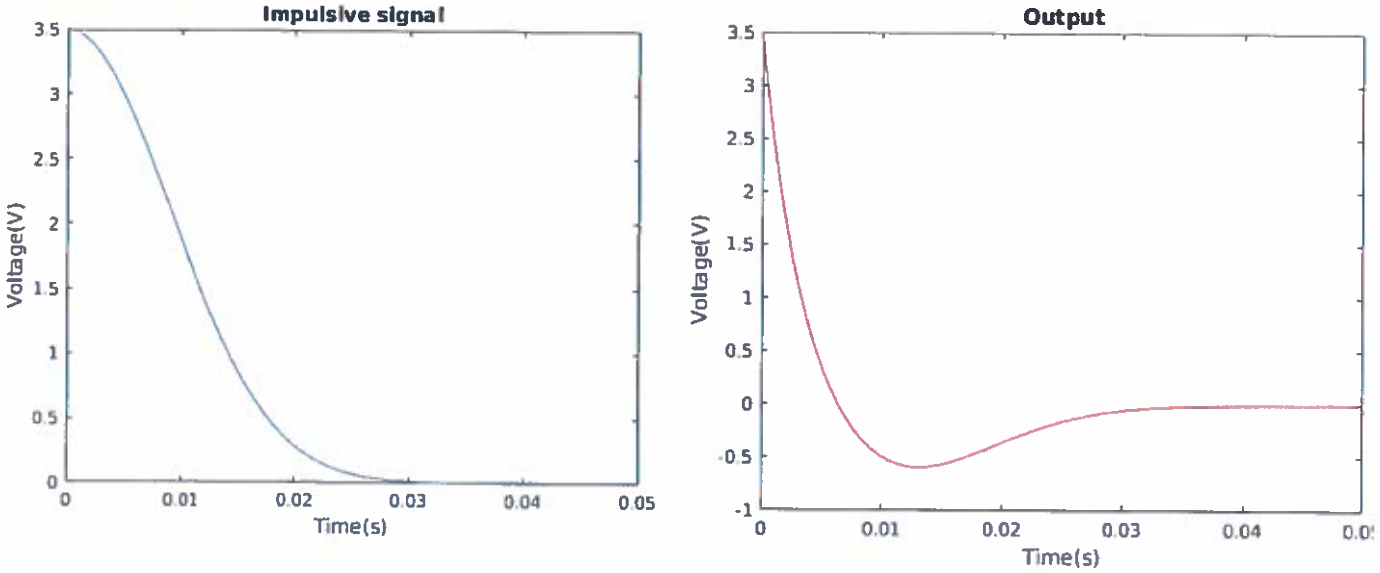


Figure 9

Ralston's method:

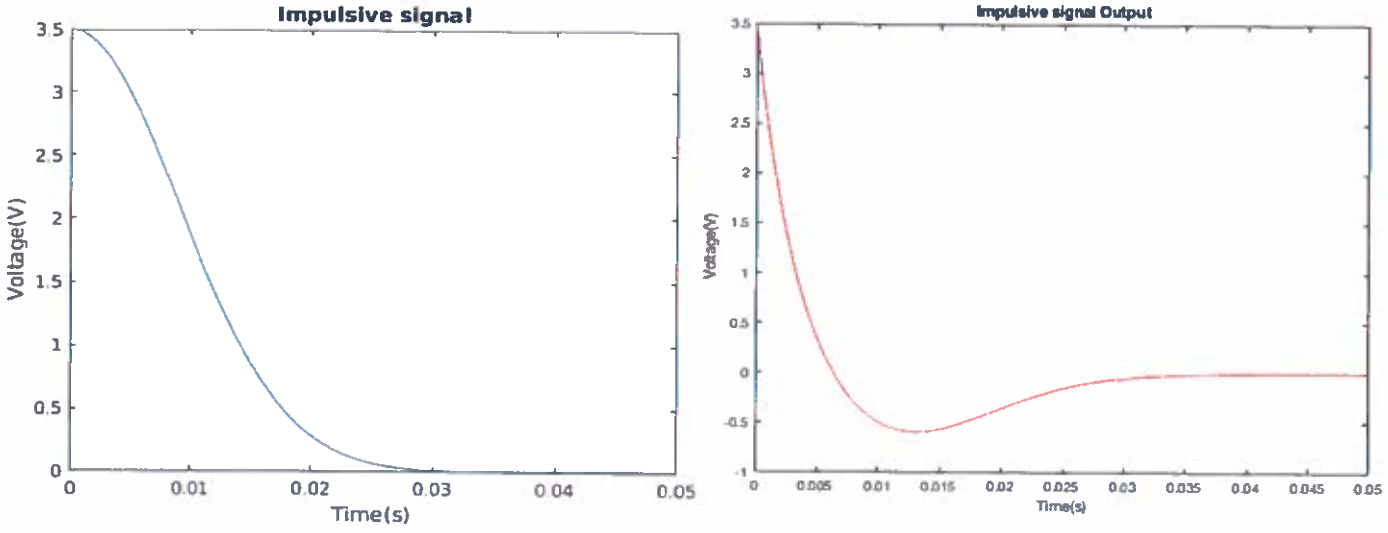


Figure 10

Midpoint method:

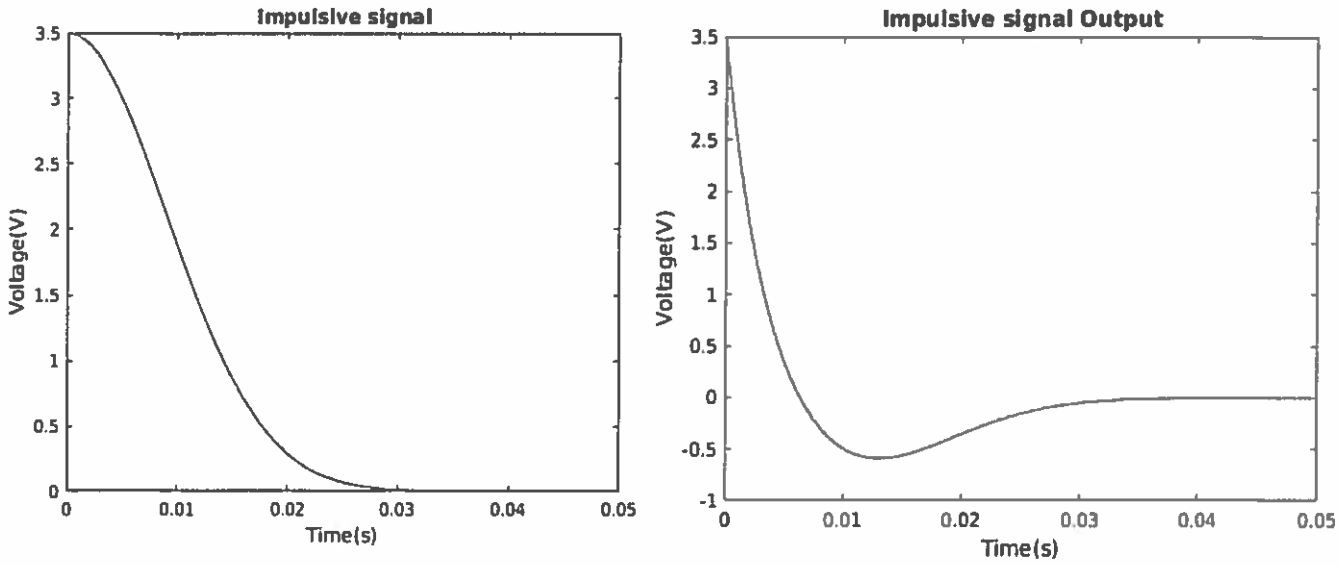


Figure 11

For the decay signal, there is a noticeable dip towards a minimum point (around -0.5) before returning to 0. This is most likely due the inductor in the RL circuit – when the current through an inductor is changed, the inductor resists the change by producing a voltage between the leads in opposing polarity to the change.
In the following section, the results from the impulsive decay signal can be seen.

Heun's method:

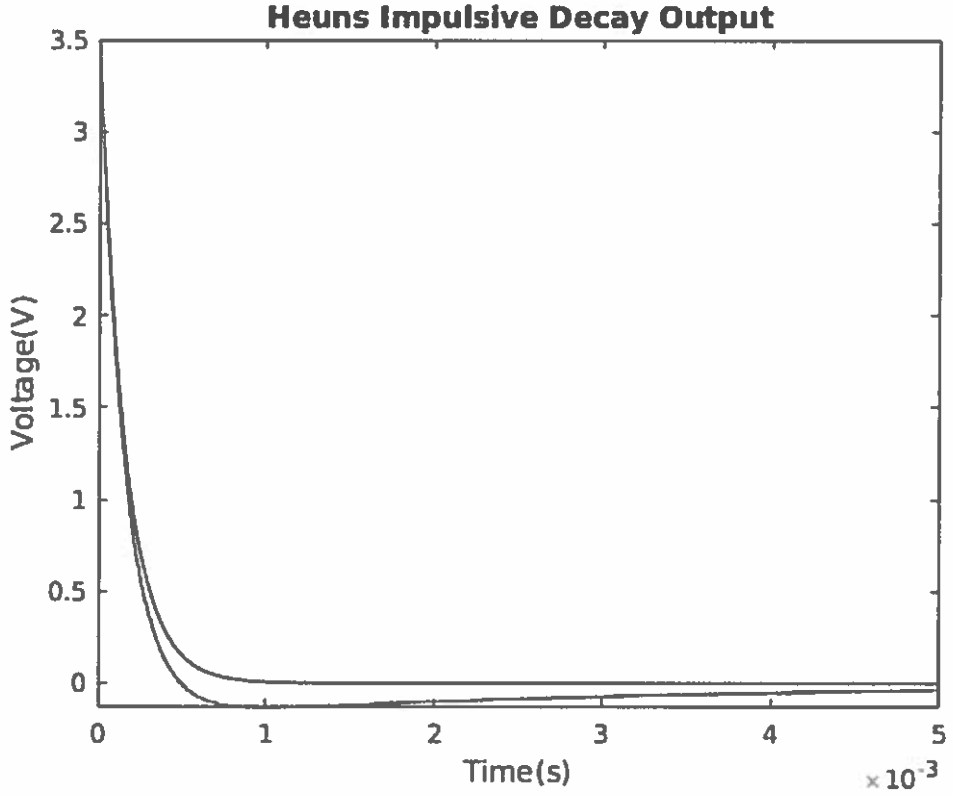


Figure 12

Ralston's method:

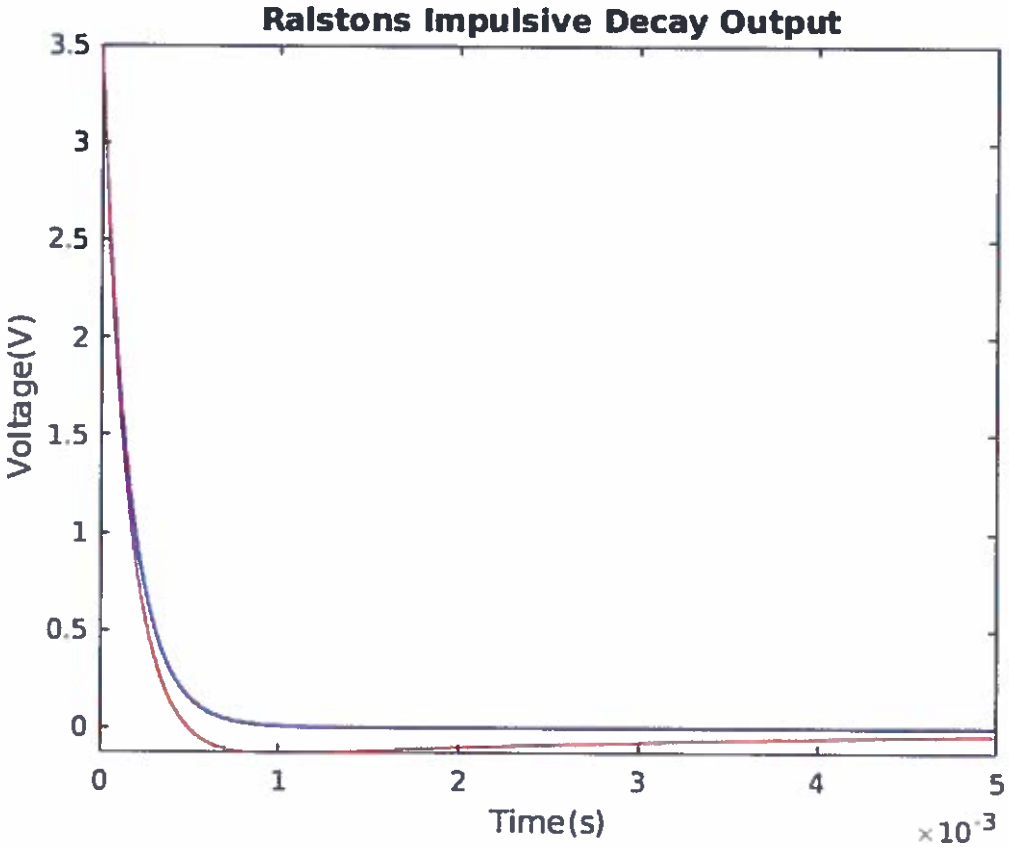


Figure 13

Midpoint method:

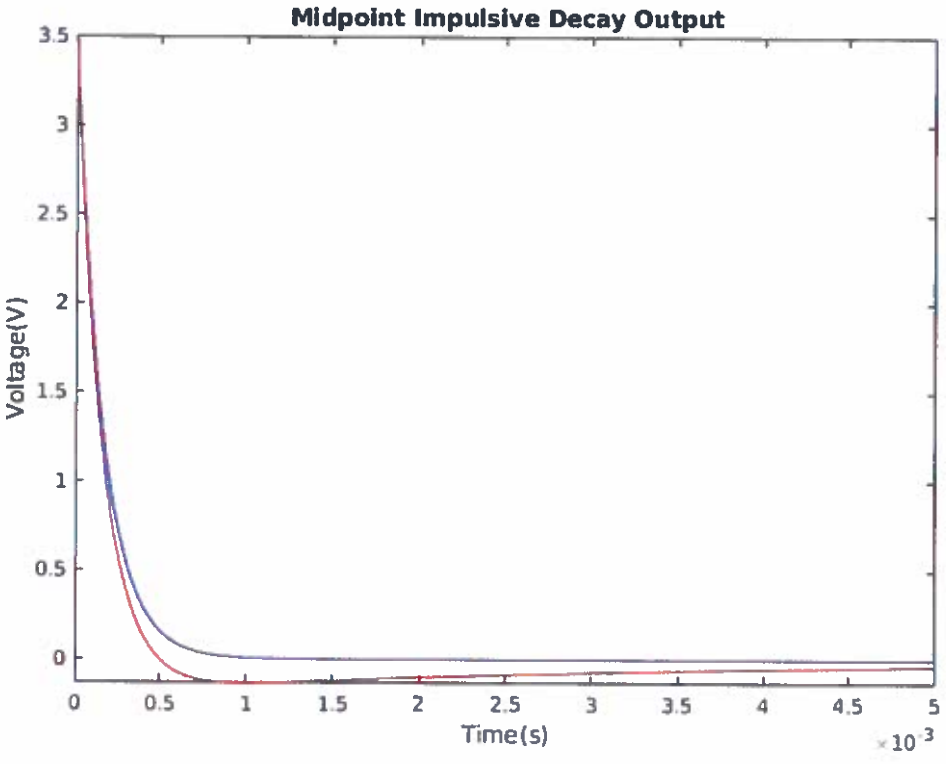


Figure 14

Here the output signal is clearly an impulsive signal, which can be seen in all the plots in figures 12-14. The output starts at $3.5V$ and it decays unlike the prior step function. This is because the input is decreasing, so it is expected that the output be as well. The resistor in the circuit eventually dissipates the energy, which is why the output eventually reaches zero.

1.3 Sine wave at variable frequencies

The next inputs were all sinewaves with amplitudes of 4.5V and frequencies as described in table 1.

Table 1

Time (μs)	Frequency (Hz)
20	50,000
160	6250
450	2222.2222
1000	1000

In the following section the results can be observed. The **red** lines indicate output and the **blue** lines indicate input.

Heun's method:

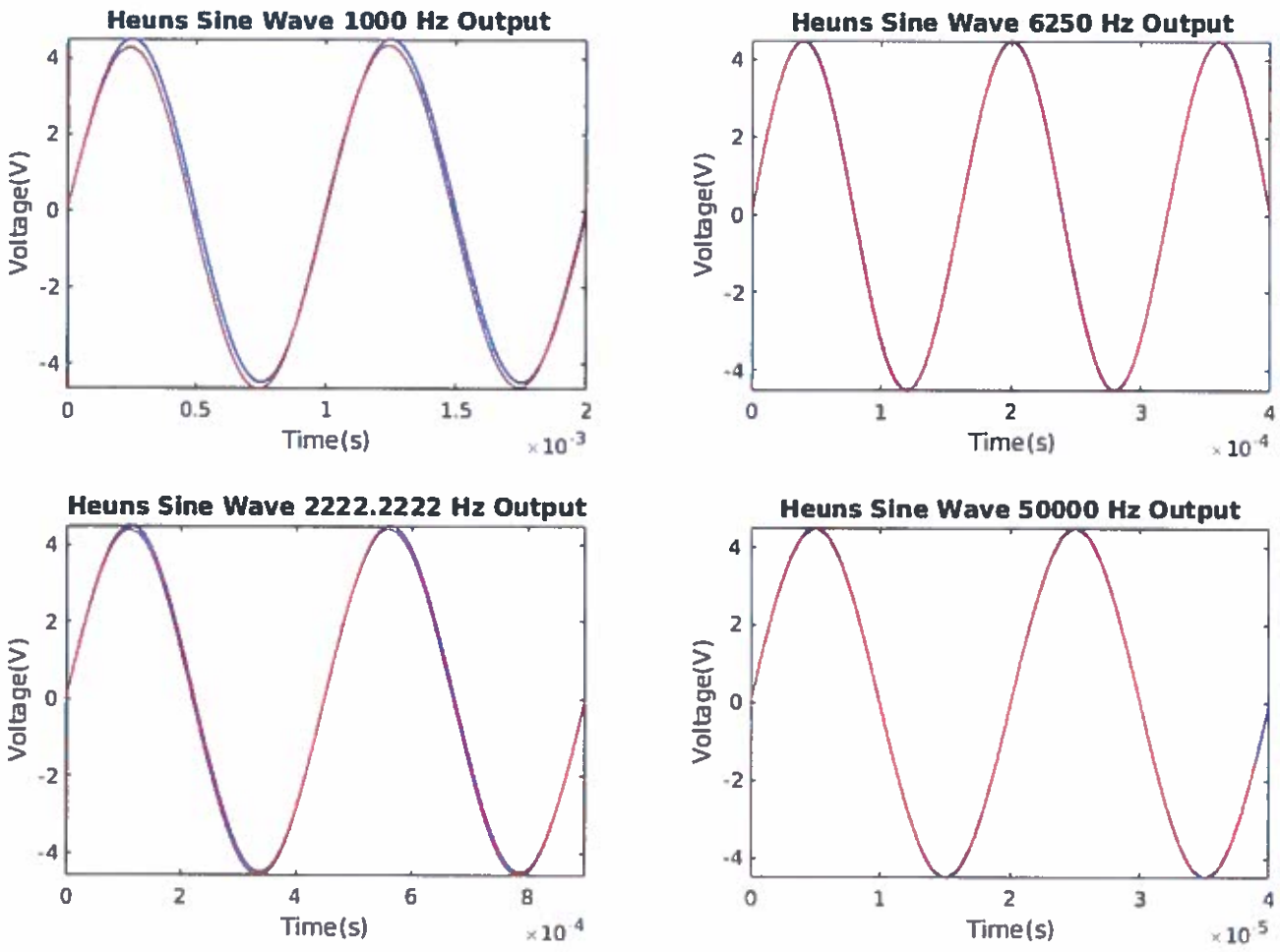


Figure 15

Ralston's method:

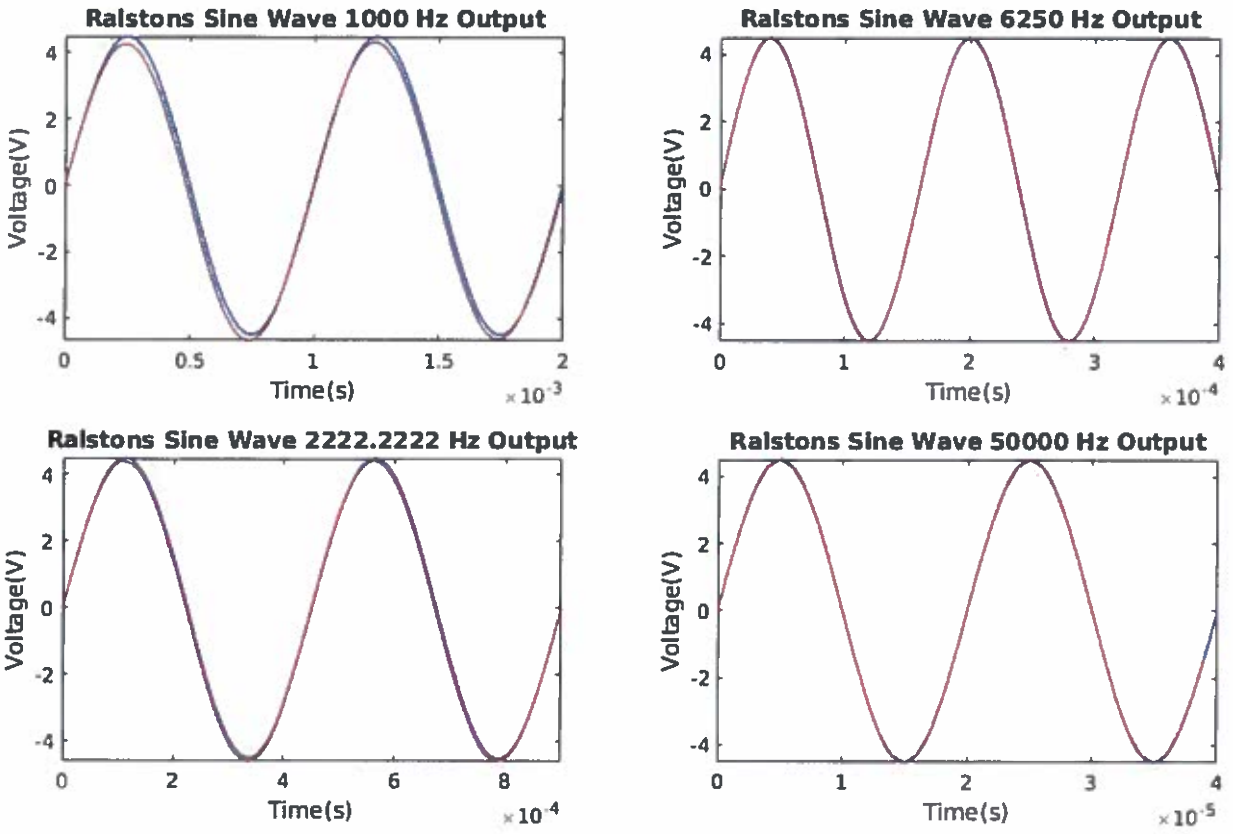


Figure 16

Midpoint method:

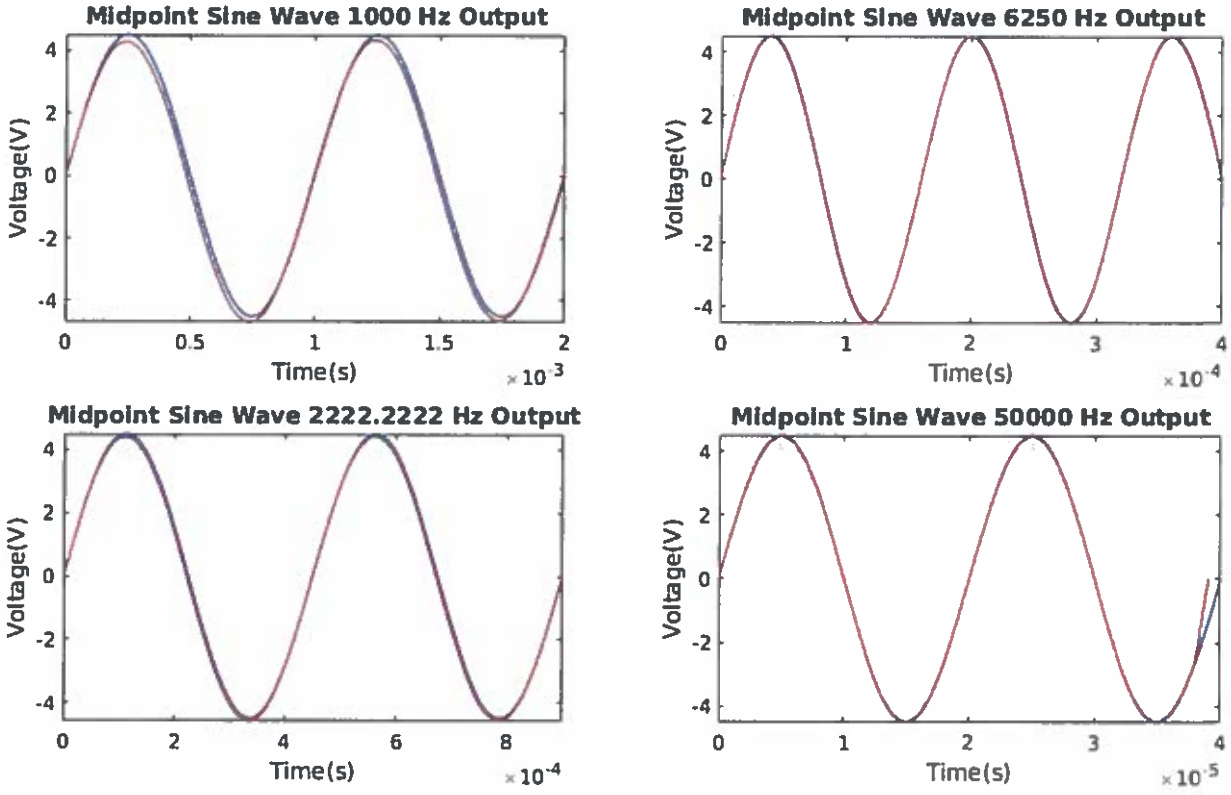


Figure 17

As the frequency increases, notice that the output (almost) matches the input signal. On the other hand, as the frequency decreases the difference between our numeric method and the input increases.

This as expected, as the circuit is a high pass filter, meaning low frequencies will be blocked and high frequencies will pass through. This is shown from the slight attenuation in the sine wave output for $1000Hz$, this is the result of the high pass filter.

1.4 Square wave with variable frequency

The square wave was implemented using the built-in MatLab function 'square'. All square waves had amplitude 4.5 and frequencies as described in table 2.

Table 2

Time (μs)	Frequency (Hz)
20	50,000
160	6250
450	2222.2222
1000	1000

Heun's method:

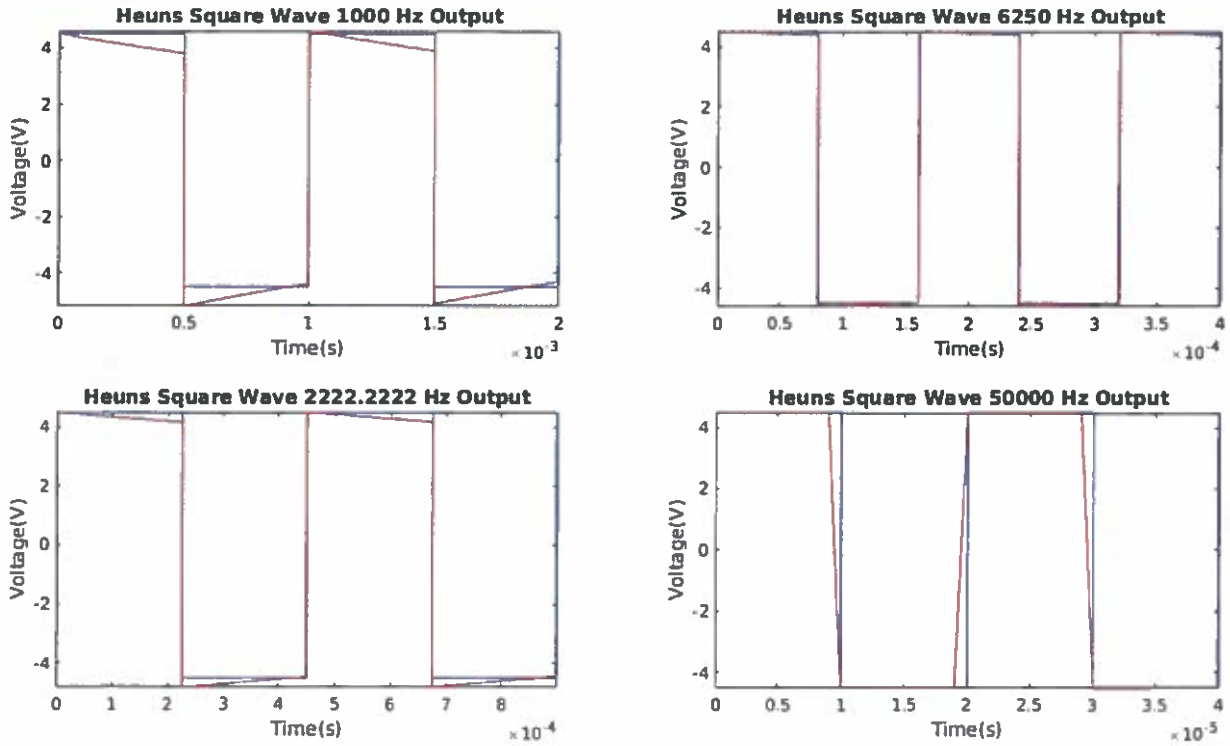


Figure 18

Ralston's method:

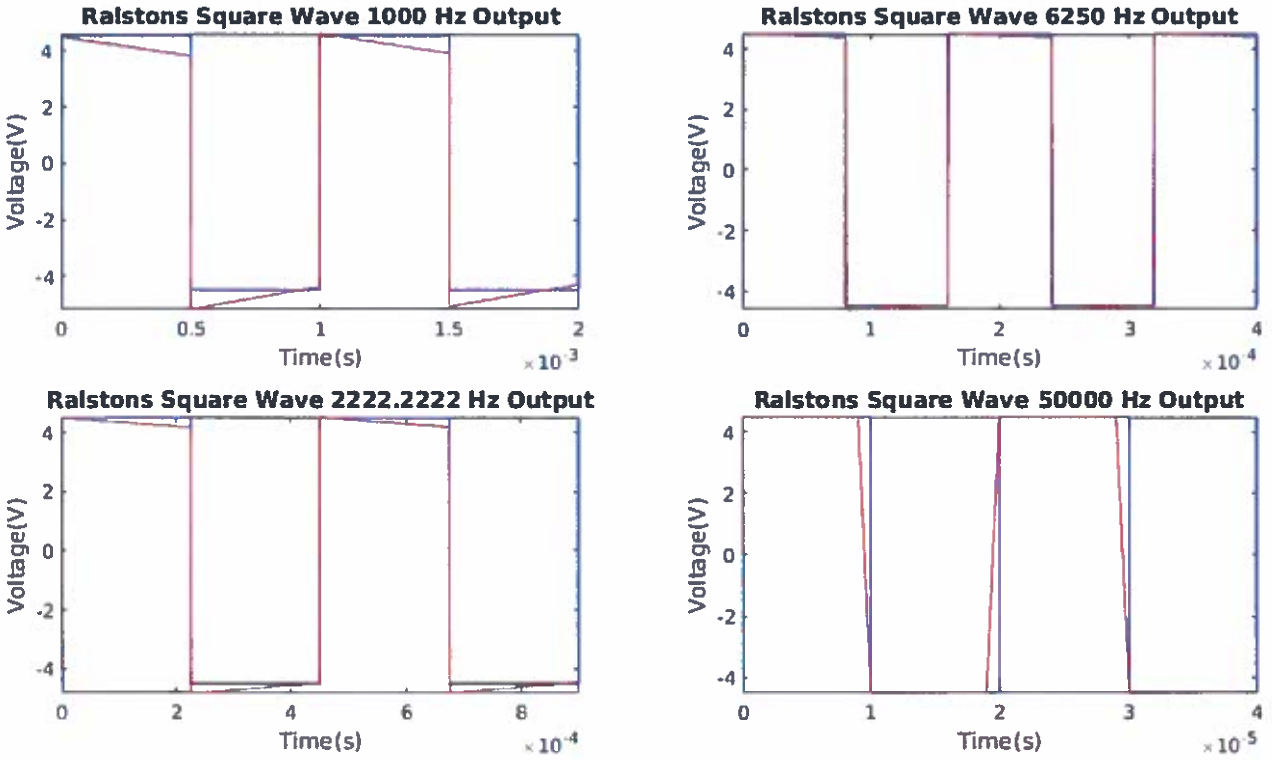


Figure 19

Midpoint method:

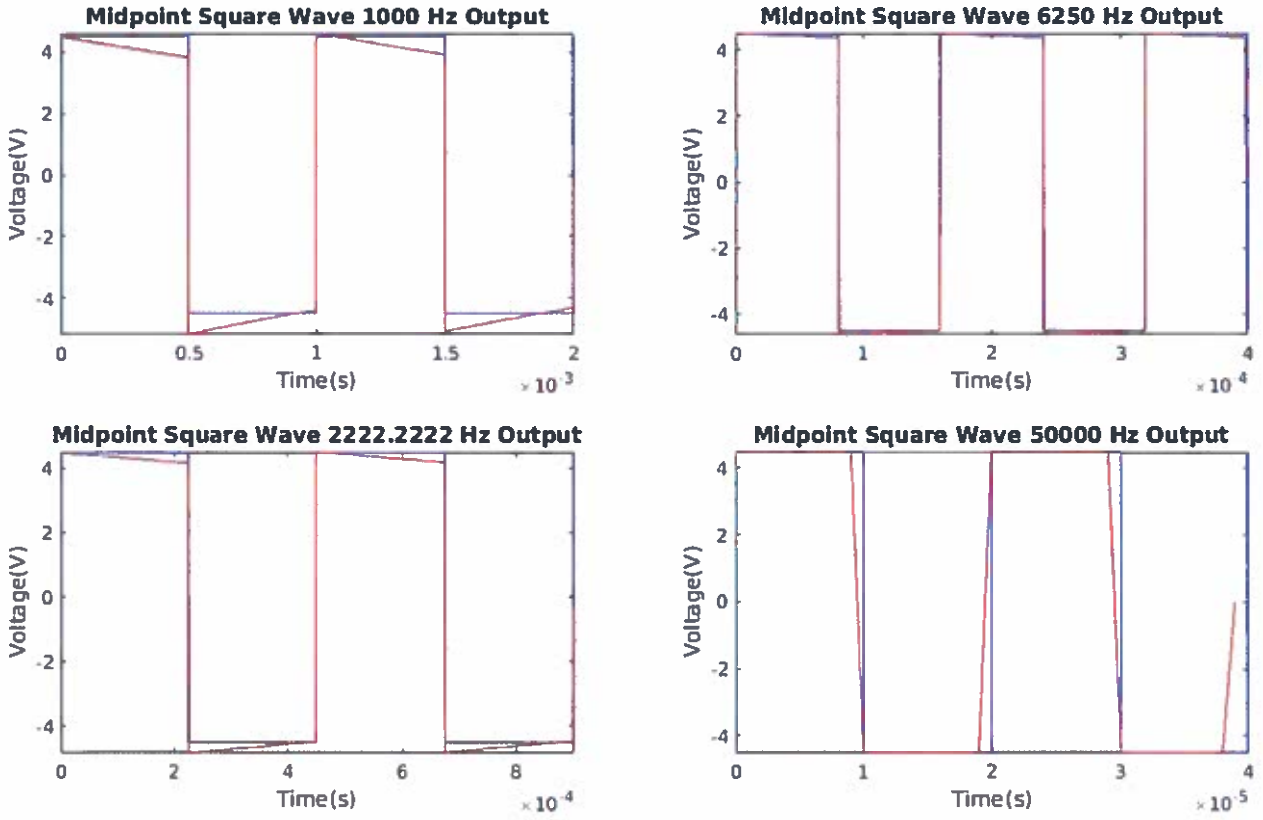


Figure 20

Attenuation of the output signal increases as frequency is decreased. This is due to the high pass component of the circuit, as stated previously.

However, notice that at $50kHz$ the gradient of the output signal becomes tilted rather than straight; one reason for this could be that the current follows the voltage in a RL circuit, giving a slight phase shift in the output signal.

1.5 Sawtooth function with variable frequencies

The final input was the sawtooth function having an amplitude 4.5 and frequencies as described in table 3.

Table 3

Time (μs)	Frequency (Hz)
20	50,000
160	6250
450	2222.2222
1000	1000

See the output below. Input is marked in blue, and output in red.

Heun's method:

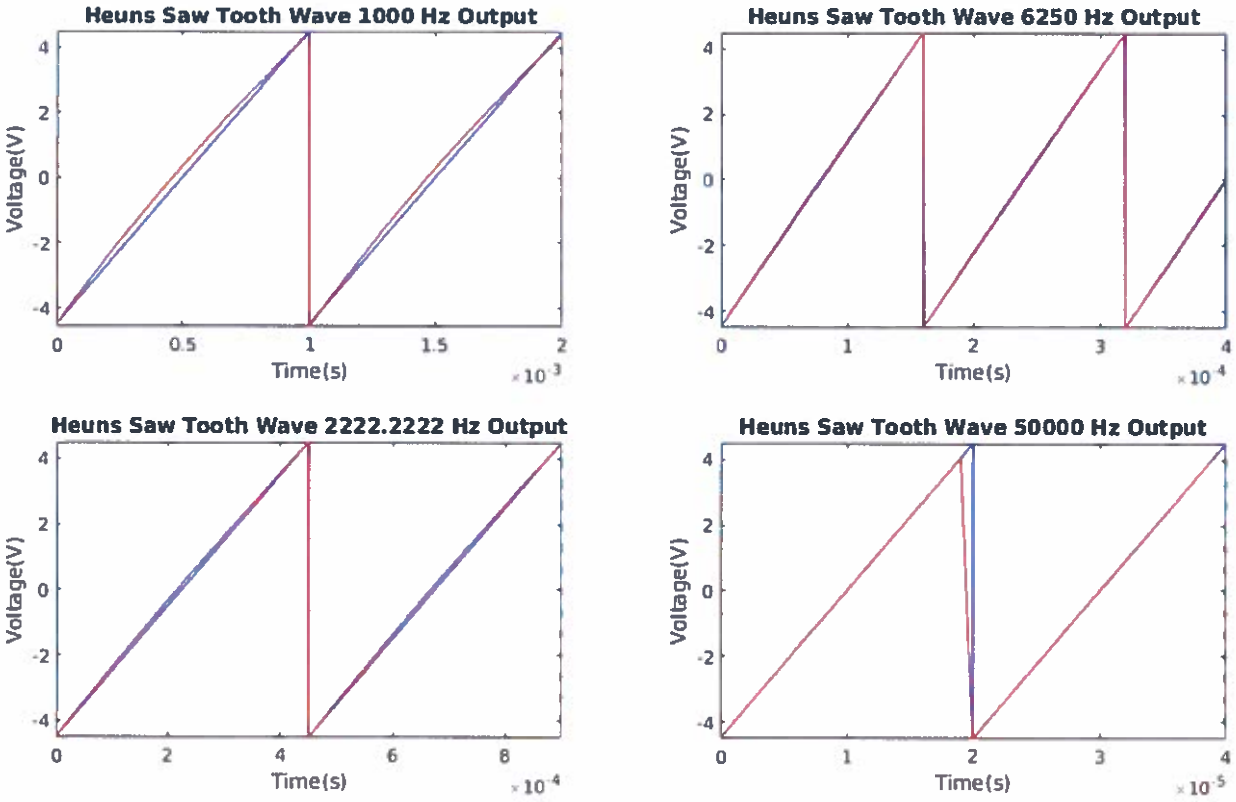


Figure 21

Ralston's method:

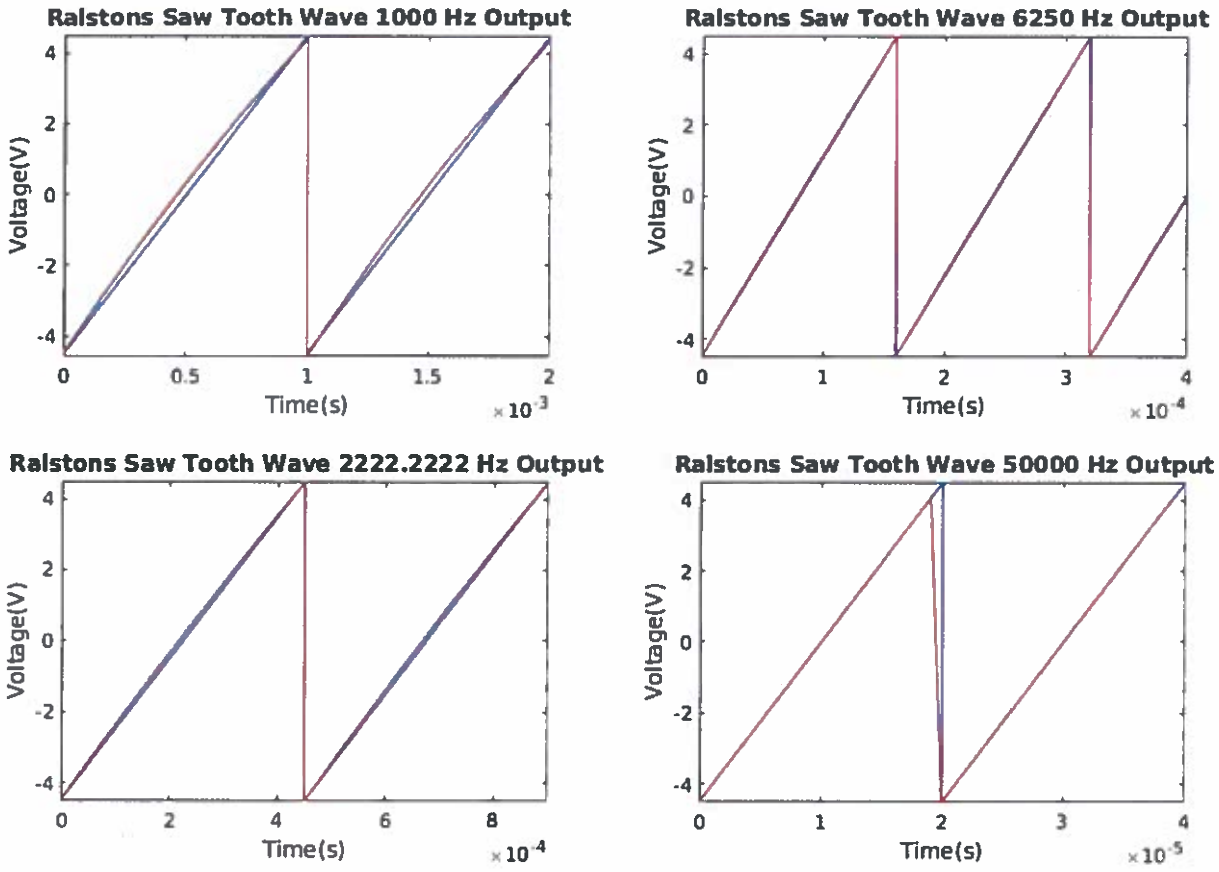


Figure 22

Midpoint method:

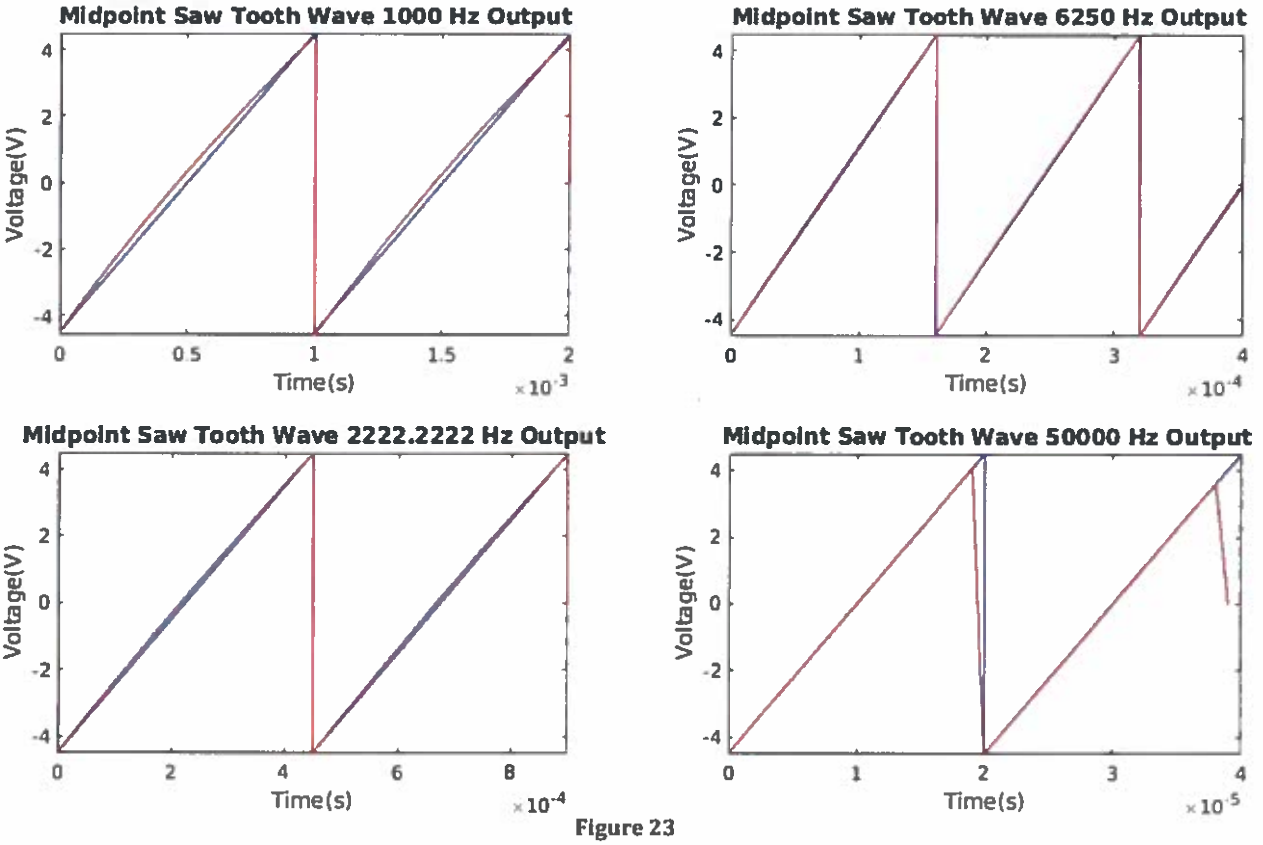


Figure 23

In the sawtooth wave there is little attenuation of the signal -this could be due to the sharp change in the signal due to the saw tooth function. Notice that at the highest frequency ($50kHz$) there is once again a slight deviation to the input signal. Here it is clear that the output signal is lagging behind the voltage, most likely due to the current following the voltage in a RL circuit.

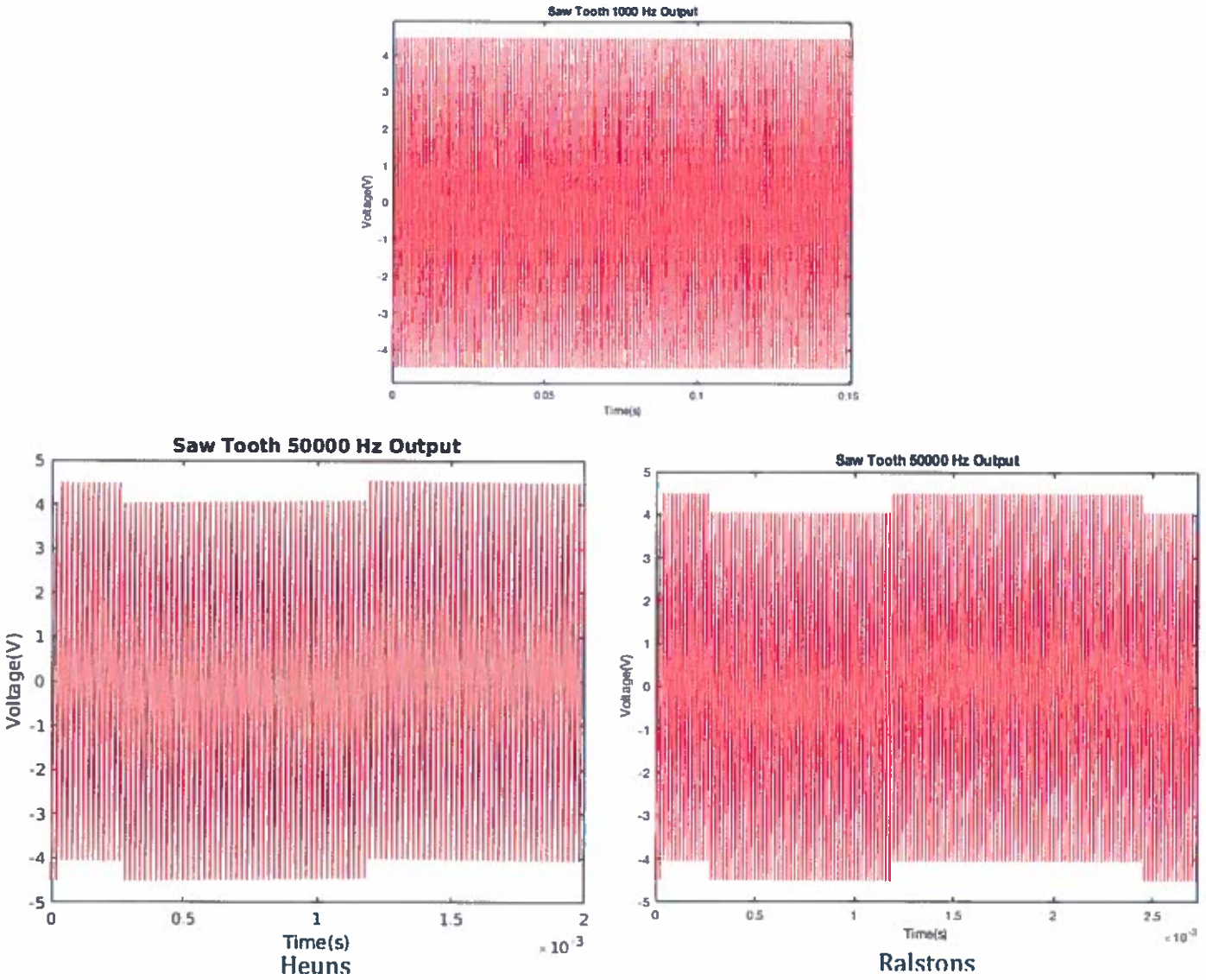


Figure 24

The amplitude of the output of the sawtooth function is attenuated periodically, especially at higher frequencies. This can be seen when increasing the time range on the plot. At $50000Hz$, there is a significant change in amplitude periodically, while at $1000Hz$, the amplitude remains relatively constant in comparison.

2. Error calculation

Exercise 2. Write a script called **error_script.m** in which you carry out error analysis given as input a cosine wave of period $T = 150\,\mu s$ and amplitude $\tilde{V}_{in} = 6\,V$. Use your favourite method to obtain the exact solution of the ODE and compare the numerical solution with the exact solution, obtaining the error as a function of t. Plot the error function for each method. Vary the time step h for a suitable number and range of values, and obtain a log-log plot to show the order of the error. Compare the errors between the three methods, do you notice anything?

Figure 25

See figure 26 for the plot of the exact solution.

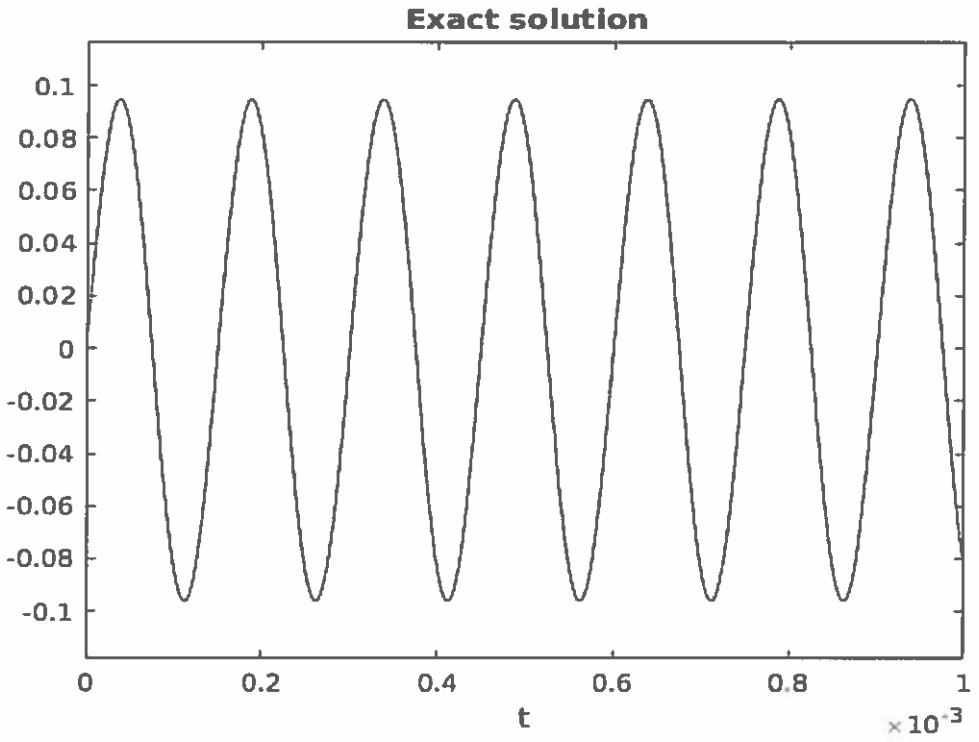


Figure 26

NB: PLOT IS OF THE EXACT CURRENT SOLUTION TO THE ODE.

Heun's error plot:

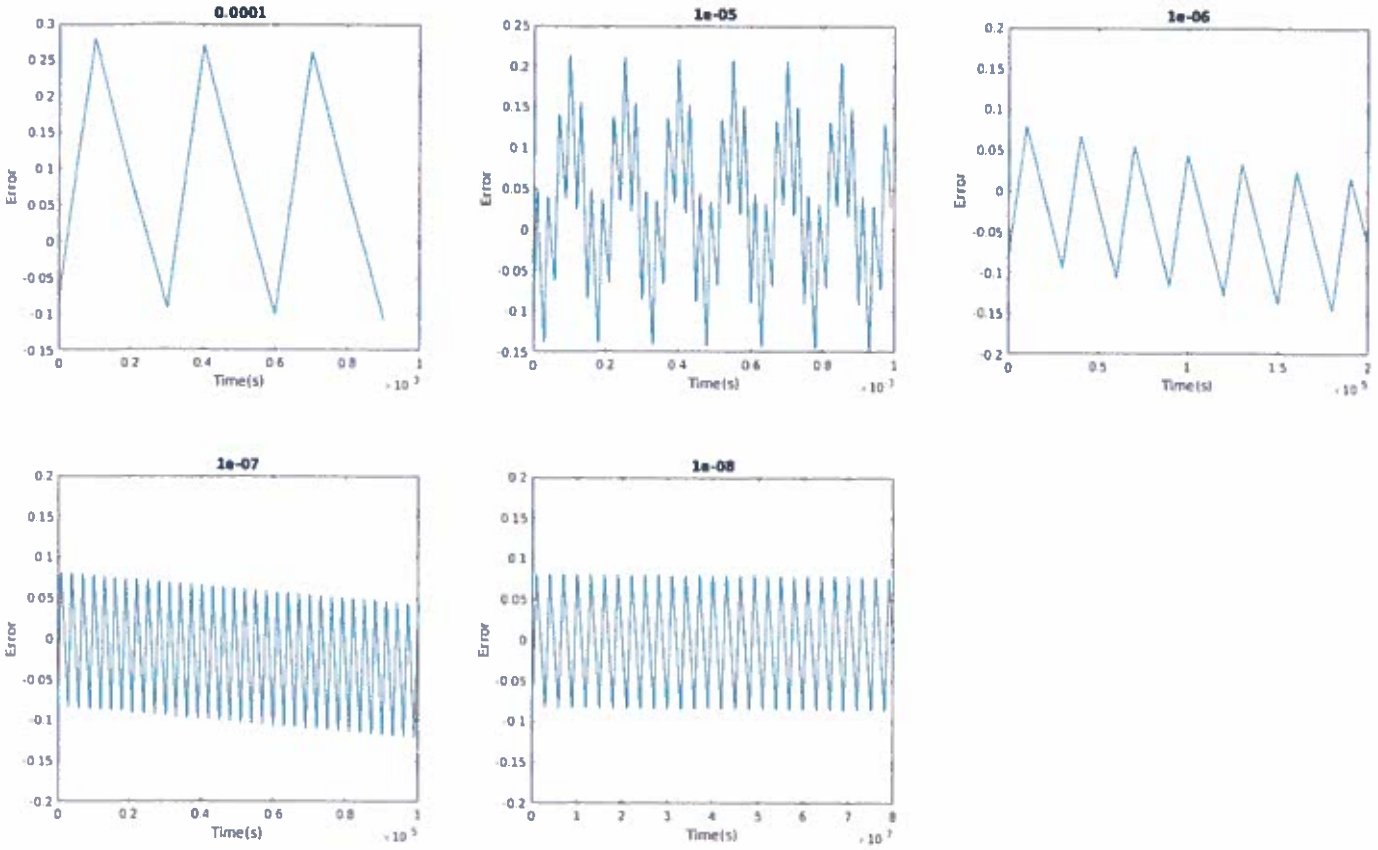


Figure 27

Table 4

(Heuns) Step size	Worst error
0.0001	1.083880e-01
1e-05	2.681114e-02
1e-06	3.397851e-03
1e-07	8.309648e-04
1e-08	5.732011e-04

Ralston's Error Plot:

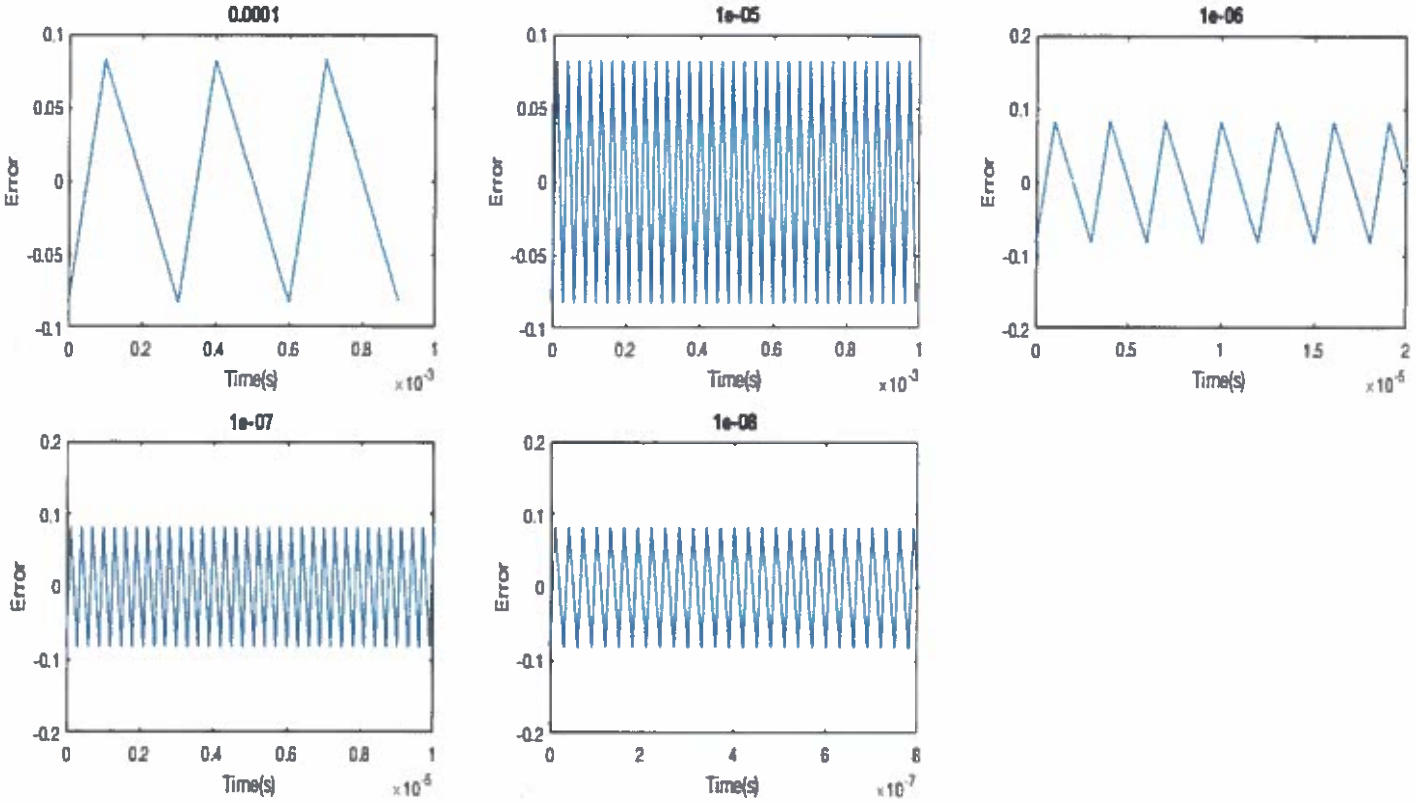


Figure 28

Table 5

(Ralstons) Step size	Worst error
0.0001 worst error	8.307397e-02
1e-05 worst error	8.293807e-02
1e-06 worst error	8.298583e-02
1e-07 worst error	8.299077e-02
1e-08 worst error	8.299120e-02

Midpoint Error Plot

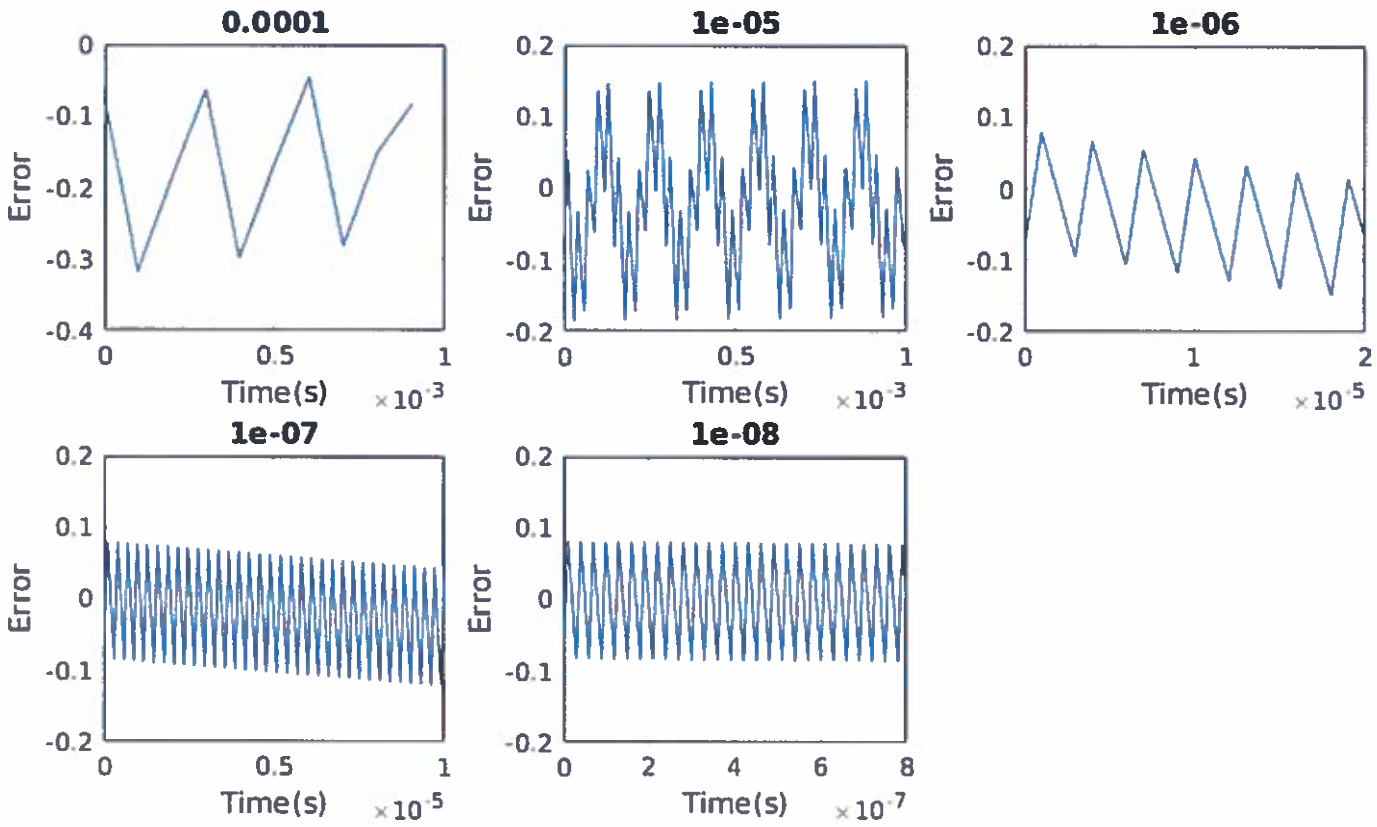


Figure 29

Table 6

(Midpoint) Step size	Worst error
0.0001 worst error	8.307403e-02
1e-05 worst error	8.307403e-02
1e-06 worst error	8.307403e-02
1e-07 worst error	8.307402e-02
1e-08 worst error	8.307395e-02

The step size was varied from $1e^{-4}$ to $1e^{-8}$, and the plots are shown in figures 27-29. It can be seen that as the step size gets smaller the output with respect to time gets more and more compressed. (Note that the time axis gets stretched out to show the signal).

Whereas step size seems to affect the maximum error for Heuns method, Ralston's method has a much smaller error at bigger step sizes, and remained under 0.1 for absolute error in the cases tested. In contrast, Heuns reaches almost 0.3 for maximum absolute error.

Next, the log-log plot for Heun's method error was obtained:

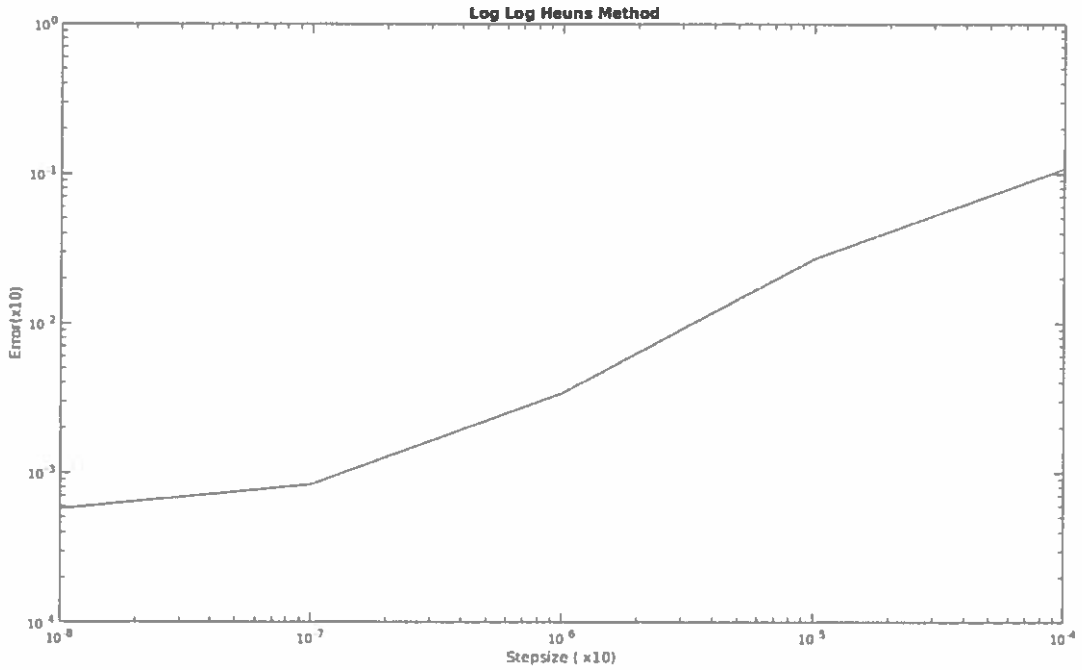


Figure 30

Then Ralston's:

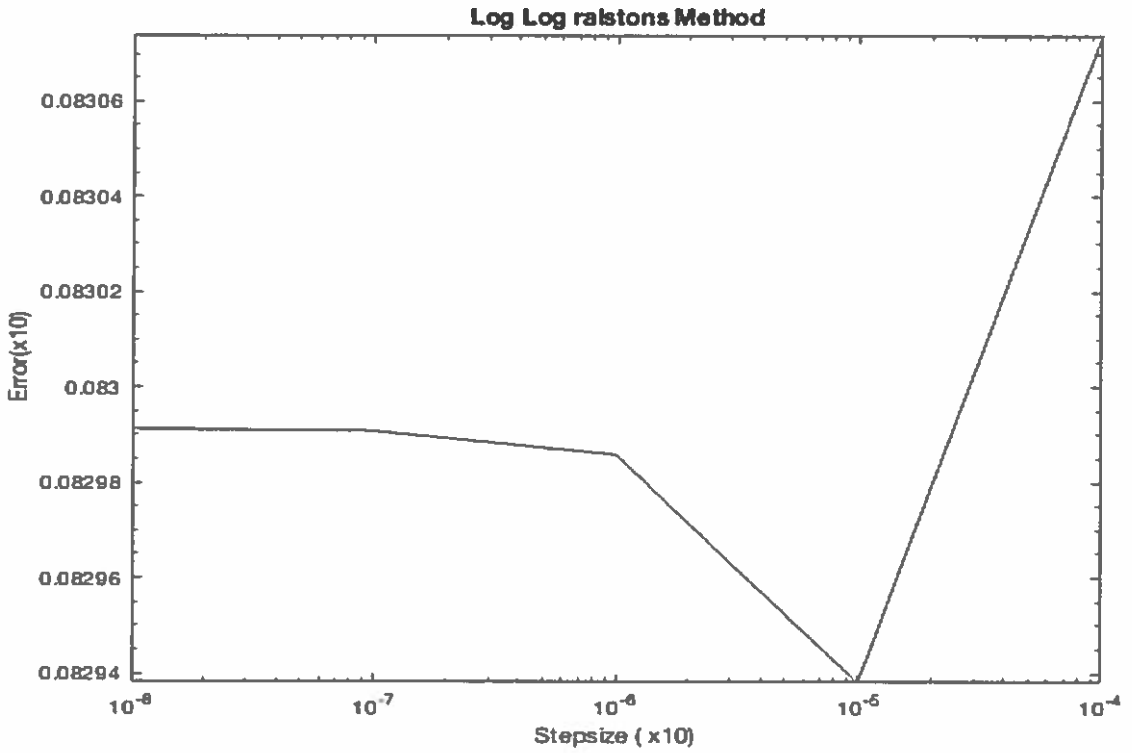


Figure 31

And for Midpoint:

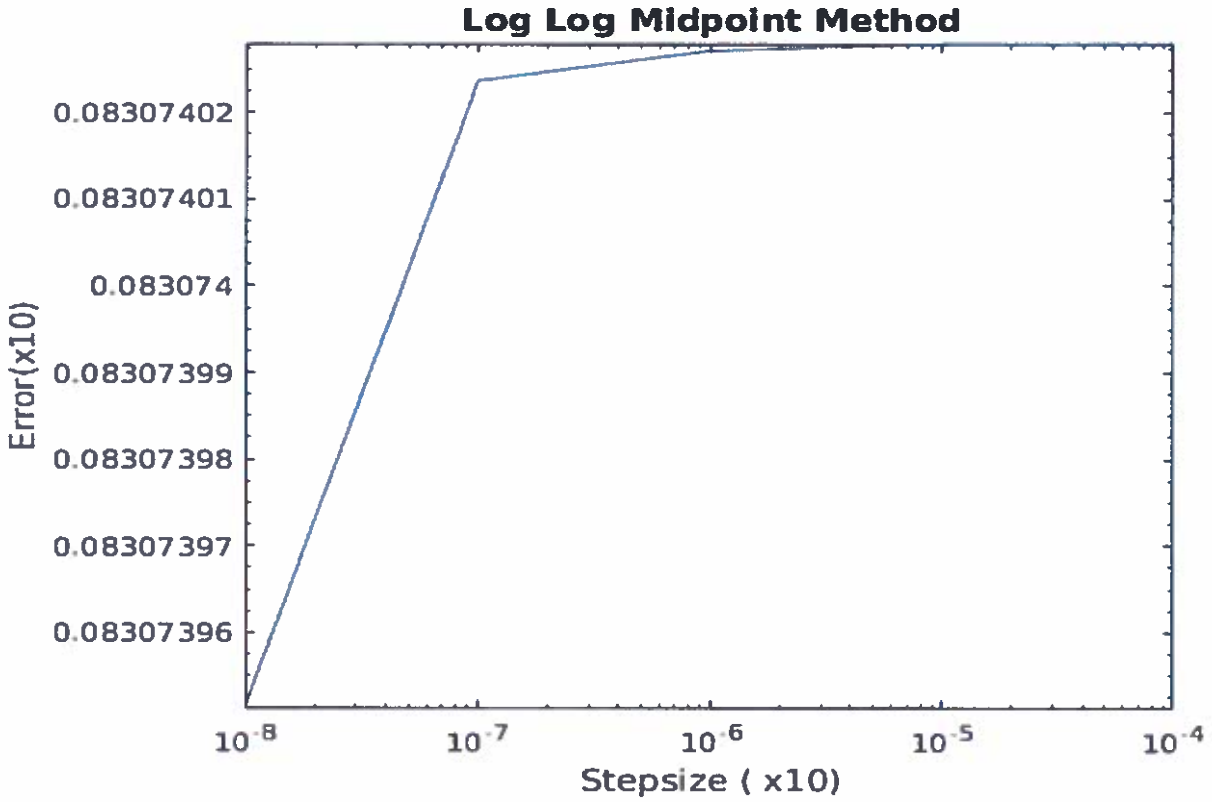


Figure 32

And finally, altogether

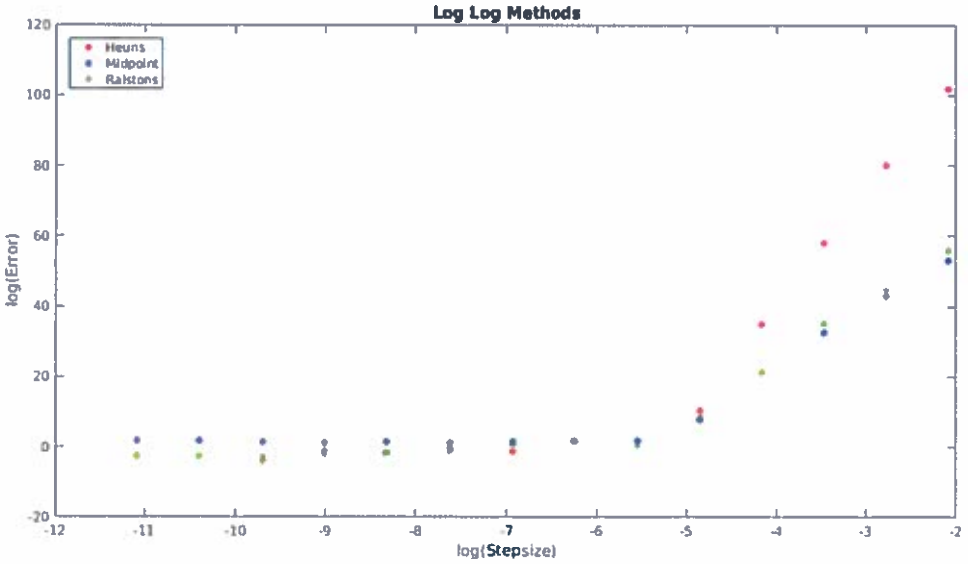


Figure 33

To obtain the log-log plot the maximum of the absolute error of the time graph was first found. Then each value was placed into an array and plotted against the h step value.

This enabled an effective comparison of the maximum error introduced for different step sizes.
Here it can be seen, that as the step size is increased, the error grows exponentially for Heun's method, which is as expected. This is because accurate approximations have a smaller step size, as this means that there are more 'chunks' that we can take thereby increasing our precision.

By comparison, Ralston's log-log plot is very focused, with a much smaller range of log error values. This indicates that the error from Heun's method is significantly more varying with step size than Ralston's method, as was seen in the error plots.

The midpoint method's log-log plot was not as good as Ralston's method, but worse than Heun's method. This is as expected as Ralston's method uses both $k1$ and $k2$ as the coefficients in the calculation, thereby making it more accurate.

Lastly, from the graph with all three methods plotted obtained it is clear that as the step size increases the order of Heun's error escalates with a higher gradient compared to Midpoint and Ralston's.

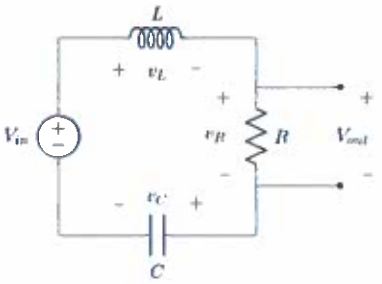
Overall our conclusion was that the order of best accuracy is

1. Ralston's Method
2. The Midpoint Method
3. Heun's Method

3. RLC Circuit

An RLC circuit represents a standard example of a harmonic oscillator, namely a device able to resonate to a sinusoidal input signal. Among the many applications for this circuit, there is the tuning of analogue radio receivers. Usually, they are used as band-pass filters.

Consider the RLC circuit as depicted in the Figure. For the circuit, we can write the following equations:



$$\begin{aligned} v_L(t) + v_R(t) + v_C(t) &= V_{in}(t) \\ L\frac{d}{dt}i_L(t) + R\,i_L(t) + \frac{1}{C}\int_0^t i_L(t) &= V_{in}(t) \\ L\frac{d^2}{dt^2}q_C(t) + R\frac{d}{dt}q_C(t) + \frac{1}{C}q_C(t) &= V_{in}(t)\,. \quad (1) \end{aligned}$$

The state is $q_C(t)$. The input is $V_{in}(t)$. The output is $V_{out} = v_R = R\frac{d}{dt}q_C(t)$. Assume that the capacitor is precharged at time $t = 0$ with $q_C(0) = 500\,\text{nC}$. Moreover, no initial current flows through the inductor at time $t = 0$: $i_L(0) = \frac{d}{dt}q_C(0) = 0\,\text{A}$. The values for resistance, capacitance, and inductance are:

$$R = 280\,\Omega\,, \quad C = 4\,\text{pF}\,, \quad L = 600\,\text{mH}\,.$$

Figure 34

3.1 Step function input

The first input introduced was a Step function with amplitude 5V. See figure 35 for the result.

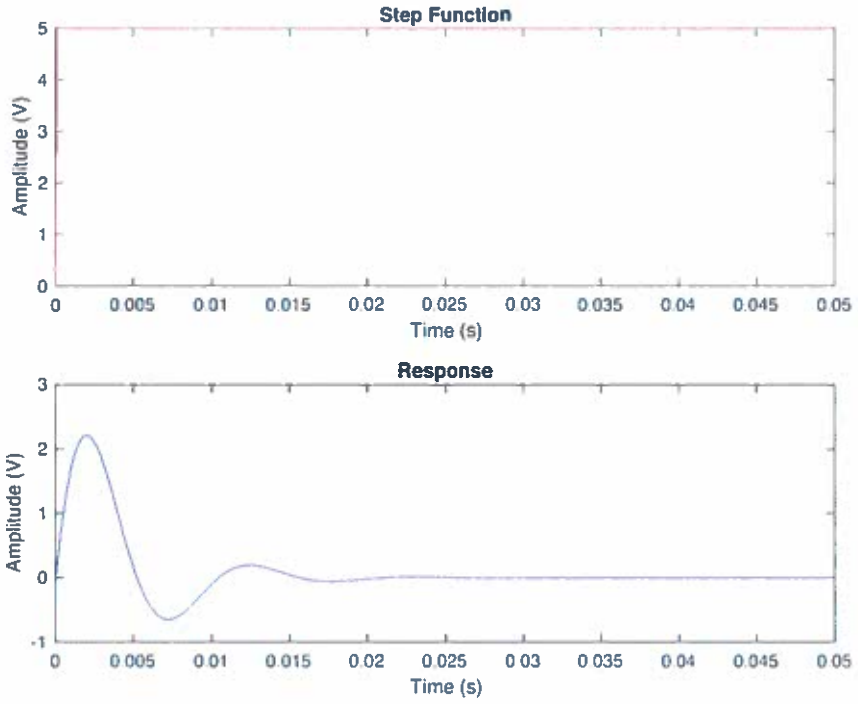


Figure 35

The response for an RLC circuit to a unit step function is an oscillating output voltage decaying exponentially. This is because initially the passive components see an infinite frequency, but the current across the components cannot change instantaneously. So the charge sloshes between the inductor and capacitor until after a few microseconds, in this case 20 milliseconds, the capacitor is fully charged. Since the input voltage has

zero frequency at this point, the inductor behaves as an open circuit, whilst the capacitor behaves as an open circuit. Since the capacitor is charged to $5V$, the voltage difference across the terminals of the resistor is zero.

3.2 Decaying step function input

The second input introduced was a Decaying step function with initial amplitude $5V$ and with decay $Vin(e^{-\frac{t^2}{T}})$ where $T = 3\,ms^2$. See figure 36 for the result.

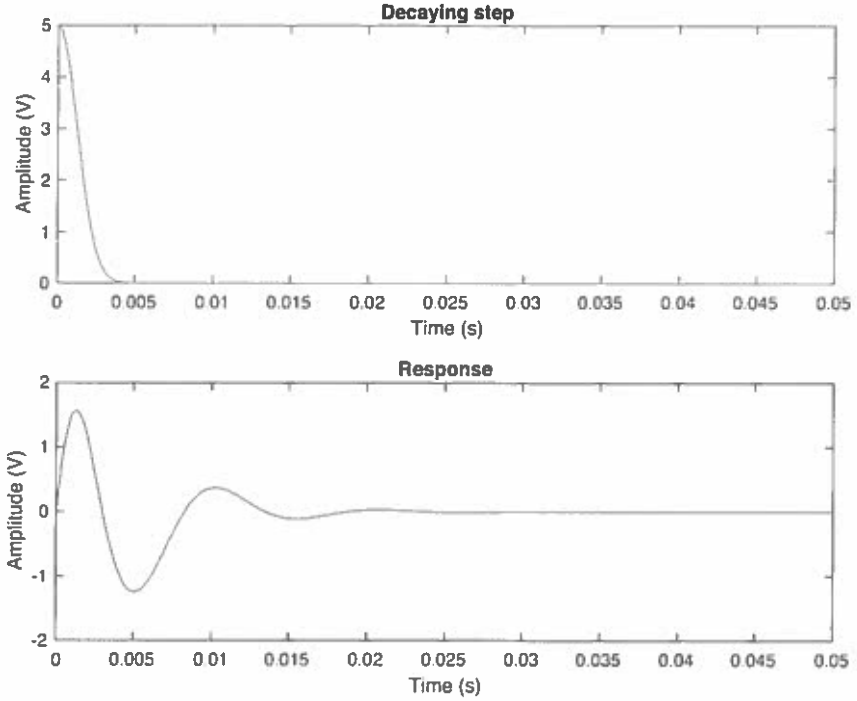


Figure 36

The response for an RLC circuit to a unit step function is an oscillating output voltage, which decays to zero. This is because initially the passive components see an infinite frequency, but the current across the components cannot change instantaneously. The reason for the large negative amplitude response at .005 seconds is because the capacitor is now charged, and no input is produced, thus meaning a larger negative value is seen across the resistor. Eventually all the charge is dissipated by the resistor, meaning the voltage across the resistor and indeed across the entire circuit becomes zero.

3.3 5Hz square wave function input

The third input introduced was a $5Hz$ square wave function with amplitude $5V$. See figure 37 for the result.

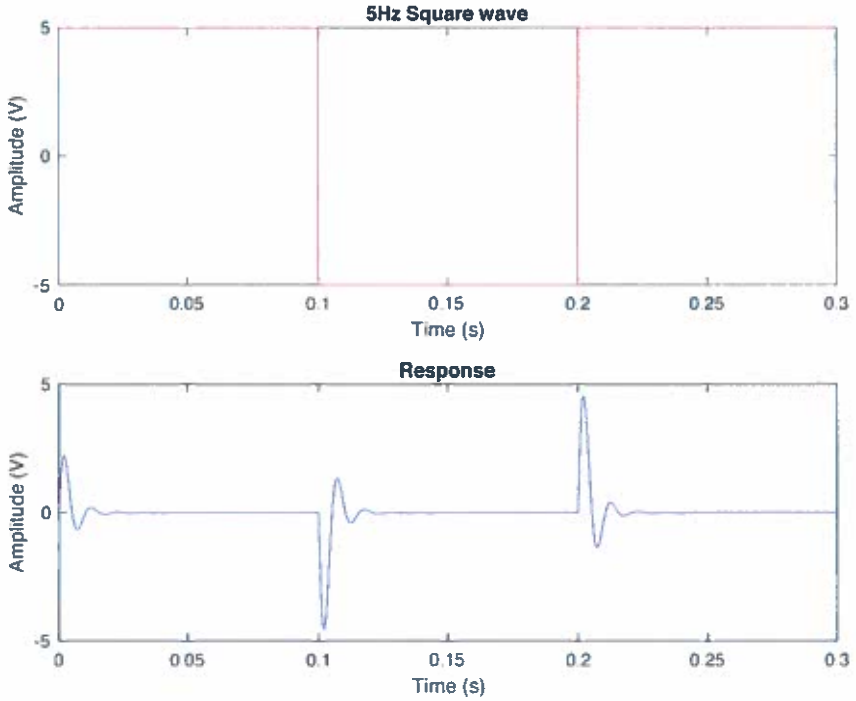


Figure 37

Initially the response is the same as for the step input function, as the frequency is so low, that by the time the input flips, the charge of the capacitor and inductor has already been dissipated by the resistor. The reason the amplitude is so large for response at .1s and .2s, is because the circuit reacts to the change in voltage rather than the actual amplitude. Therefore, when the input goes from $5V$ to $-5V$, a response with double the amplitude of the step function response can be observed because the instantaneous change is $10V$ rather than $5V$. The reason for the output amplitude being so low, is that the impedance of the capacitor is much higher than that of the inductor.

3.4 110Hz square wave function input

The fourth input introduced was a $110Hz$ square wave function with amplitude $5V$. See figure 38 for the result.

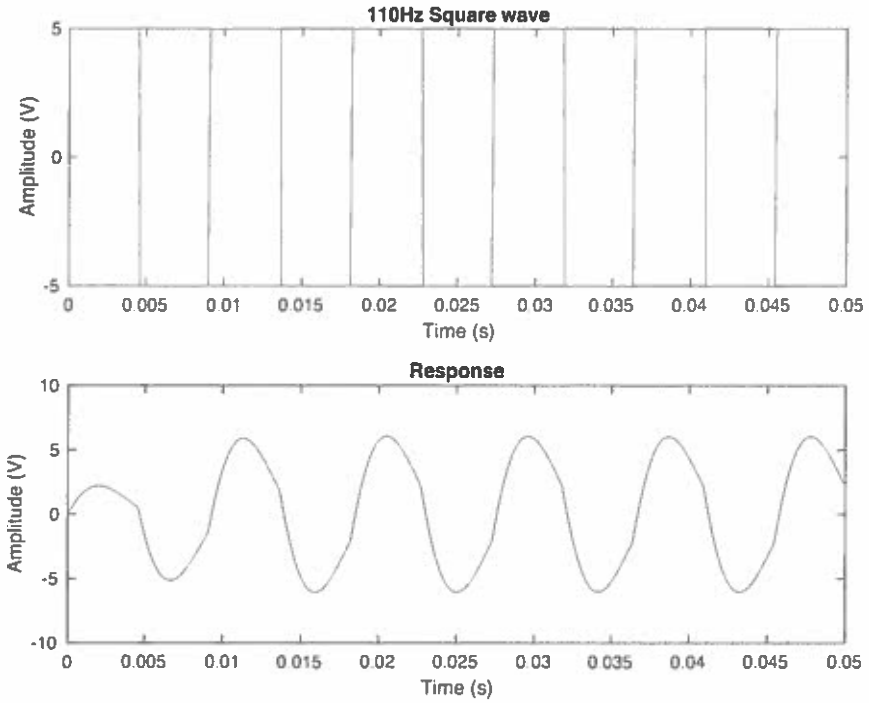


Figure 38

Here the response observed is because of the charging and discharging of the circuit. In addition, this is also the resonance frequency of the circuit, which can be observed from the response voltage being the same as that of the input. This is because at this frequency, the impedance of the inductor and capacitor cancel each other out.

3.5 500Hz square wave function input

The fifth input introduced was a $500Hz$ square wave function with amplitude $5V$. See figure 39 for the result.

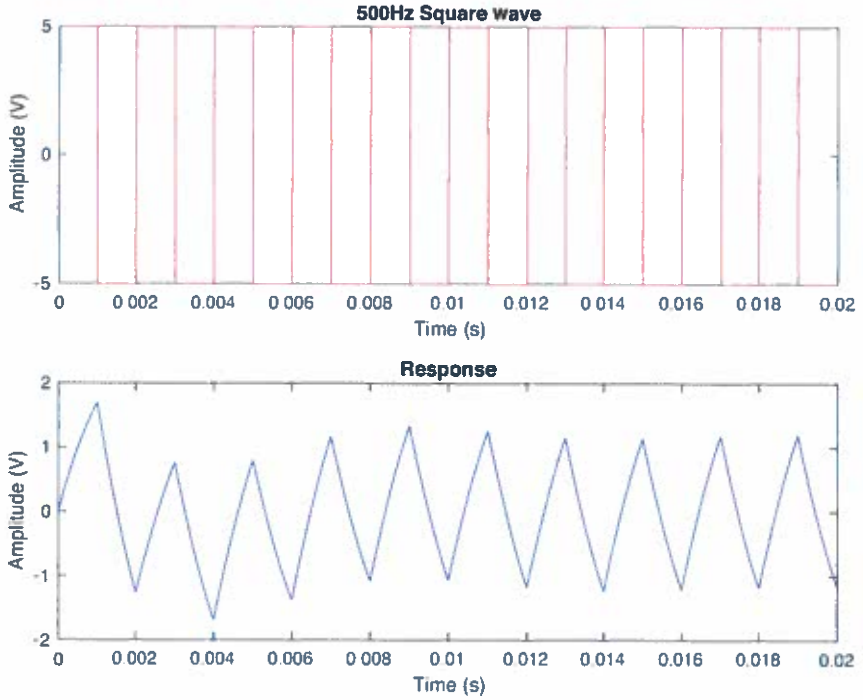


Figure 39

Here the response observed is because of the charging of the circuit only. The response looks like a triangular wave because by the time the square wave flips, the circuit has not finished charging, yet is still forced in the opposite direction because of the change of sign in the input. In addition, it can also be observed that the response also seems to have the same effect as observed from the step input. The first few of the triangles are slightly offset from zero, yet eventually normalizes as in the step response.

3.6 5Hz sine wave input

The sixth input introduced was a $5Hz$ sine wave function with amplitude $5V$. See figure 40 for the result.

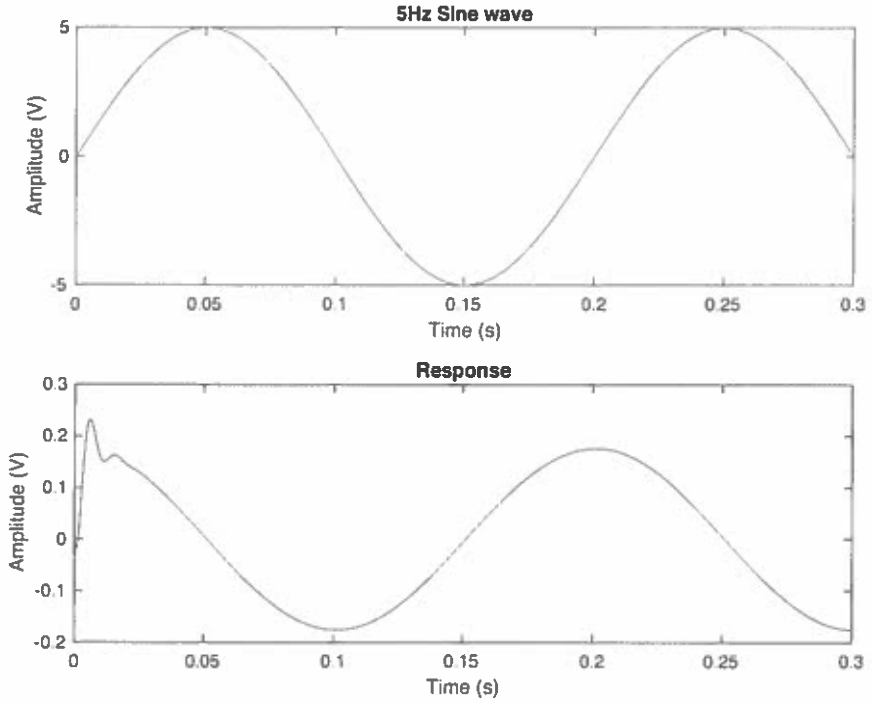


Figure 40

The initial response here is because the voltage across the capacitor cannot instantaneously change. Eventually the circuit normalizes and then the response follows the input. The gain is however incredibly low, because the impedance of the capacitor is so high. Therefore, there is barely any voltage drop across the resistor. In addition, there is also introduced a phase change in the response, because of the capacitor.

3.7 110Hz sine wave input

The seventh input introduced was a $110Hz$ sine wave function with amplitude $5V$. See figure 41 for the result.

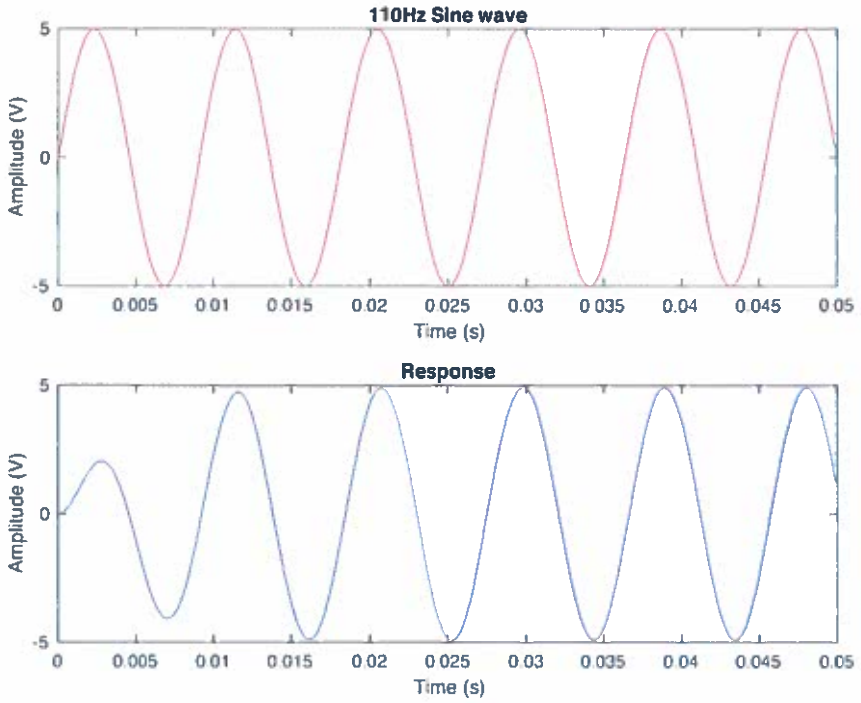


Figure 41

Here it can clearly be seen that $110Hz$ is the resonance frequency of the circuit. At first there is a response from the capacitor because it initially sees the frequency as being infinite, but after a short period of time, the circuit has stabilized, and the response hereafter clearly follows the input. This is because the impedance of the inductor and capacitor cancel each other out, and as a result, the entire voltage drop occurs across the resistor.

3.8 500Hz sine wave input

The seventh input introduced was a $500Hz$ sine wave function with amplitude $5V$. See figure 42 for the result.

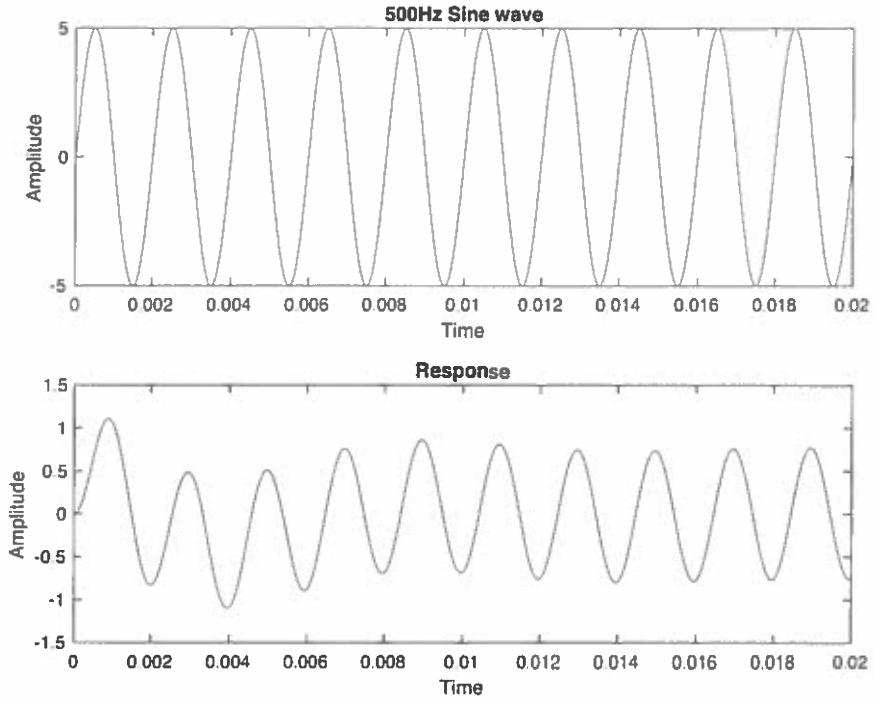


Figure 42

Here the response is reflective of the fact that the inductor now has a higher impedance than both the resistor and the capacitor. This can be seen as the voltage drop across the resistor is not very large, and in addition, a slight phase change has been introduced. The same initial response as seen in the step function can also be observed. The first few sine waves of the response are slightly offset from zero because the capacitor has to stabilize.

4. Finite Differences for PDE

The 1-D heat equation

$$\frac{\partial y}{\partial t} = \frac{\partial^2 y}{\partial x^2}, \quad 0 < x < 1, \quad t > 0$$

with zero boundary conditions $y(0,t) = y(1,t) = 0$, and initial condition $y(x,0) = y_0(x)$ can be solved numerically using the finite difference method outlined in lectures.

Figure 43

In this exercise, the analogy of temperature distribution throughout a rod will be used to give context to the arbritrary temperature values and boundary condition values, with the Arbitrary Temperature axis giving an indication of the temperature at a point throughout the rod, and the Input Function Parameter equivalent to all the points from one end of the rod to another.

The initial condition describes the temperature as it is distributed from one end of the rod to the other end of the rod at time $t = 0$, whilst the boundary conditions fix each end of the rod at a specific temperature. These boundary conditions are necessary to solve the heat equation, and it is assumed that everything beyond these points has no effect whatsoever on the temperature distribution throughout the rod. Therefore in a real-life applications it would be necessary to know the temperature at either end of the rod at all times, or for each end to be controlled using a system which keeps each end at the desired constant temperature.

For each of the various initial conditions and boundary conditions, there is a 2D plot with each line representing a different value of m, which is an index into the solution matrix and represents an arbitrary time unit from 0 to t, and a 3D plot with m displayed as a separate $Time\ (s)$ axis and converted from an arbitrary index into time in seconds calculated using the time step value k. Each of the 3D plots has been rotated for an optimal view of the behaviour of temperature throughout the rod as time progresses. The value of the temperature on the z axis is also visualised using a colour gradient from *yellow* (*hot*) to *blue* (*cold*).

In the cases where the intial condition was not equal to the boundary condition, the boundary condition is used to avoid large jumps between the $t = 0$ and $t > 0$, which would not occur in the closed form solution and makes the plots difficult to understand intuitively. The consequence of this is that the initial conditions are ignored at the boundary points and there may be large jumps in the initial conditions between the endpoints of the rods and the penultimate points at either end; however the time progression of the plots is much more intuitive to understand.

✓OK.

4.1 Tent function,

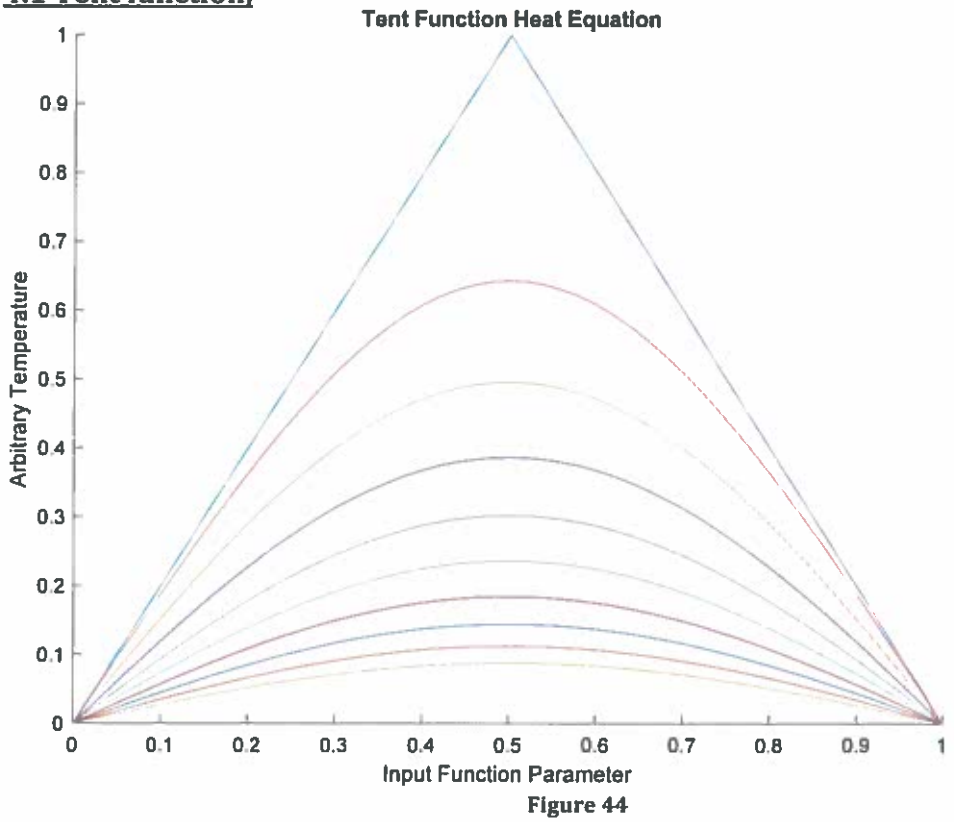


Figure 44

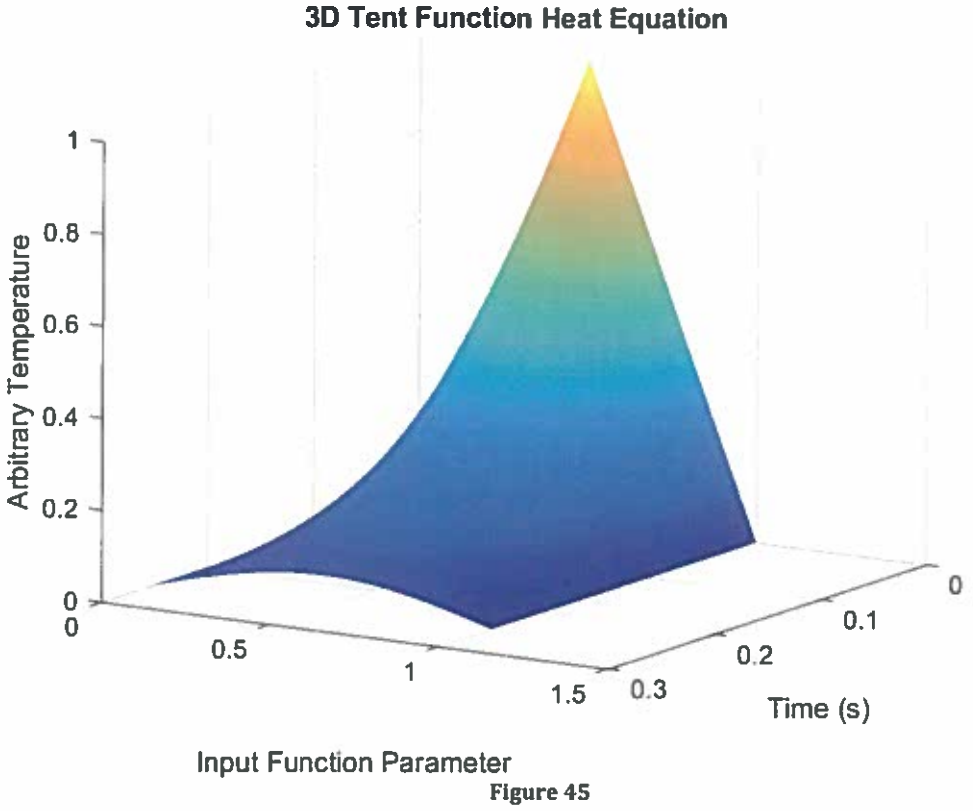


Figure 45

The tent function begins with the heat concentrated in the middle of the rod. The 2D plot, figure 44, shows in detail the line equation of heat dissipation throughout the rod

at specific points in time, whilst the 3D plot, figure 45 allows a full visualisation of the heat dissipated over time. The tent function shows an interesting property of the heat equation which is that despite the sharp point defined in the initial conditions, the line immediately smooths out and remains smooth. The overall temperature throughout the rod tends to zero since both boundary conditions are zero, meaning both ends of the rod are fixed at zero.

✓ good.

4.2 Sine function

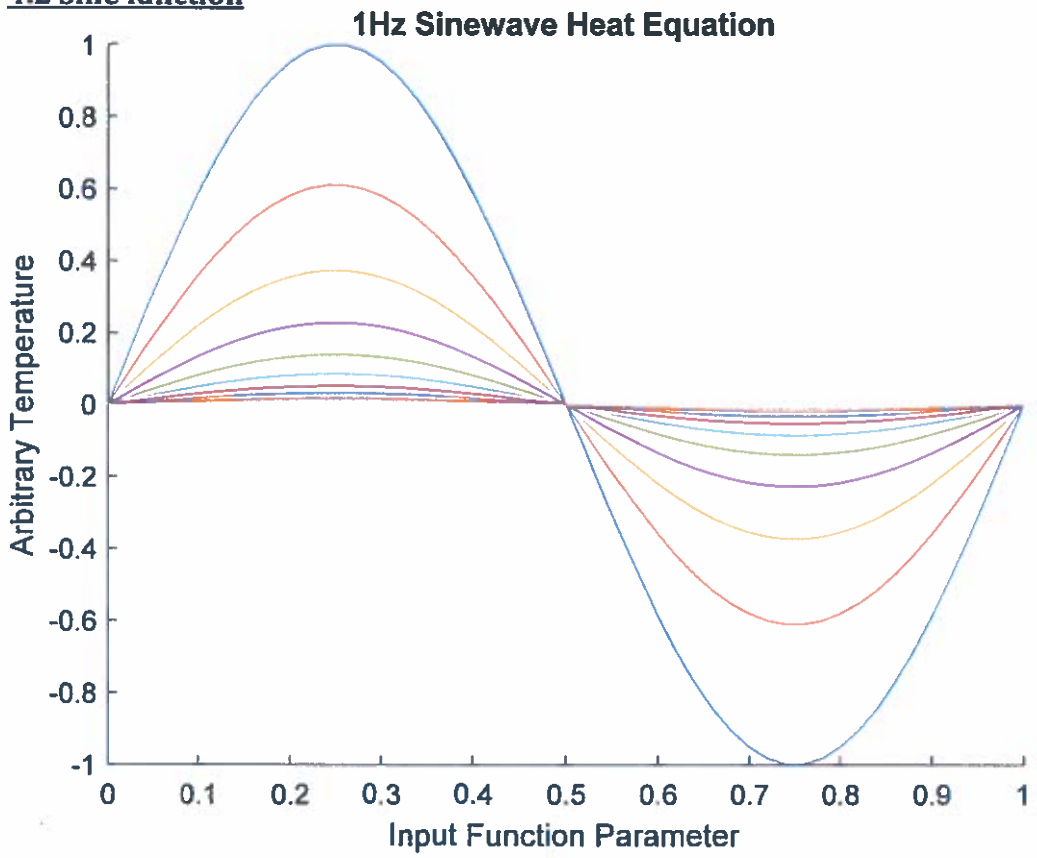


Figure 46

The total heat for the sine wave dissipated more quickly than for the tent function, since there was less energy defined in the initial condition. Hence, the time step was halved in order to show the behaviour in more detail. The 2D plot for an initial condition defined by a sinewave, figure 46, is interesting since it illustrates how in a 2D object, the temperature at a point is only influenced by adjacent temperatures. Hence, the sections from 0 to 0.5 and 0.5 to 1 have symmetry, whilst for a 3D object, and so intuitively to a human individual, the temperature differences between each side of the rod would influence each other causing the parts of the rod near the midpoint to tend to zero faster than the parts near the boundary points.

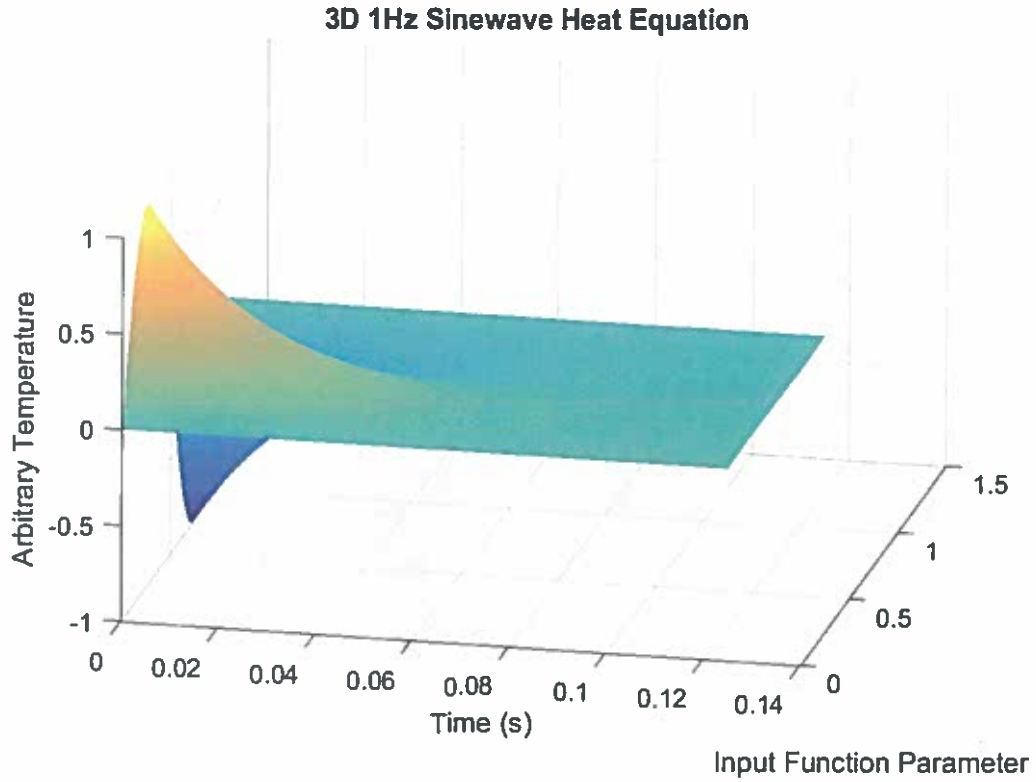


Figure 47

4.3 Abs sin function

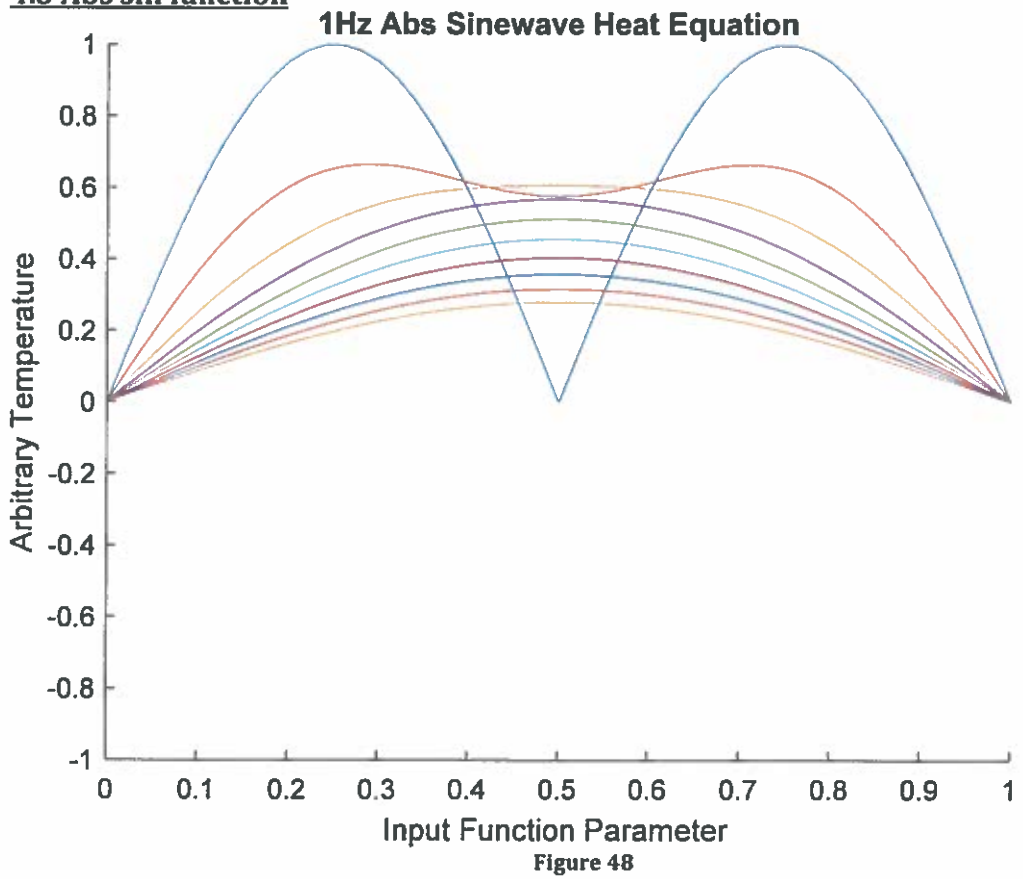


Figure 48

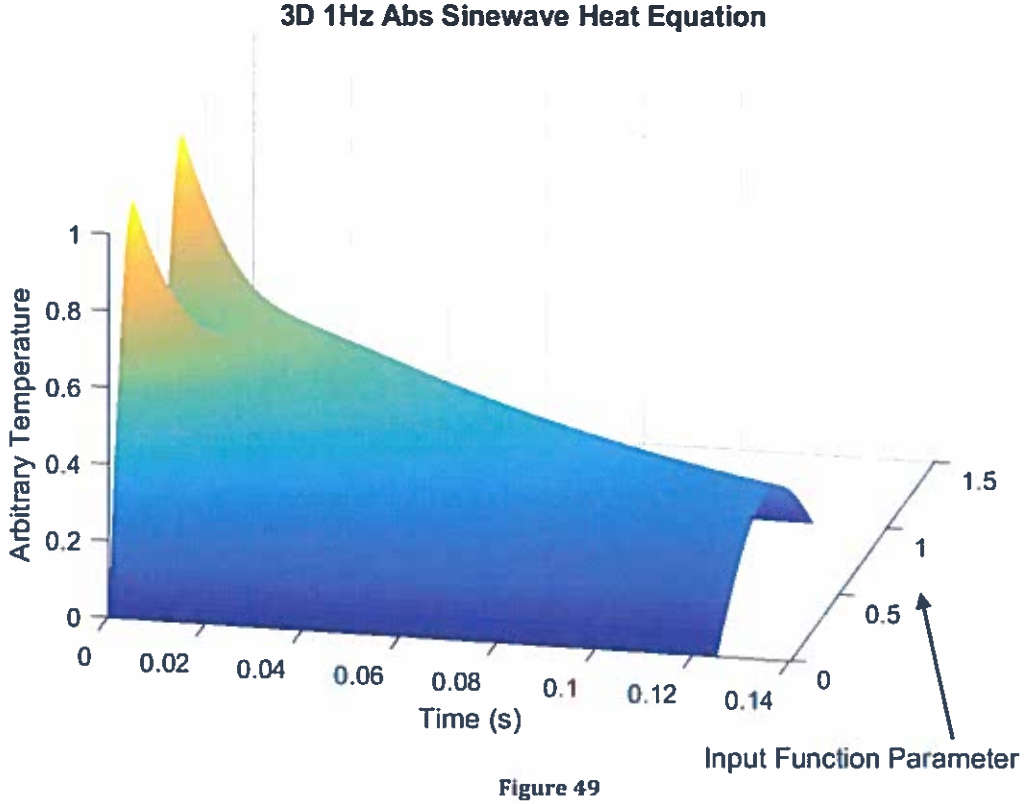


Figure 49

It is interesting to note that the absolute sine wave tended to zero temperature more slowly than the sinewave, due to having a higher starting energy. This is good sanity-check validation that the numerical analysis is producing the correct output. The 2D plot, figure 48, illustrates how heat will dissipates to remove large temperature differentials, and having differentials causes the fastest changes in temperature. The 3D plot, figure 49, illustrates the speed at which the temperature in the centre of the rod increases, shooting up to ~0.6 witin the first 0.02 seconds, before starting to decrease similar to in figure 45.

✓ good.

4.4 Sinc function – Initial condition not equal to boundary condition

Since the left boundary condition is not equal to the initial condition determined by the *sinc* function, an instantaneous jump in the temperatures between the left end point and the penultimate point on the left. This is most visible in the 2D plot, figure 50. Since the total heat for this initial condition dissipates very quickly, it was necessary to reduce the time step k by a factor of 50 in order to observe the interesting parts of the plot. Most of the energy in the *sinc* is contained in the main peak, whilst the rest of the heat energy quickly dissipates to zero. The dissipation of heat from the main peak is slightly distorted, since the left boundary is zero, causing the temperature to be lower and dissipate faster than for a closed form solution.

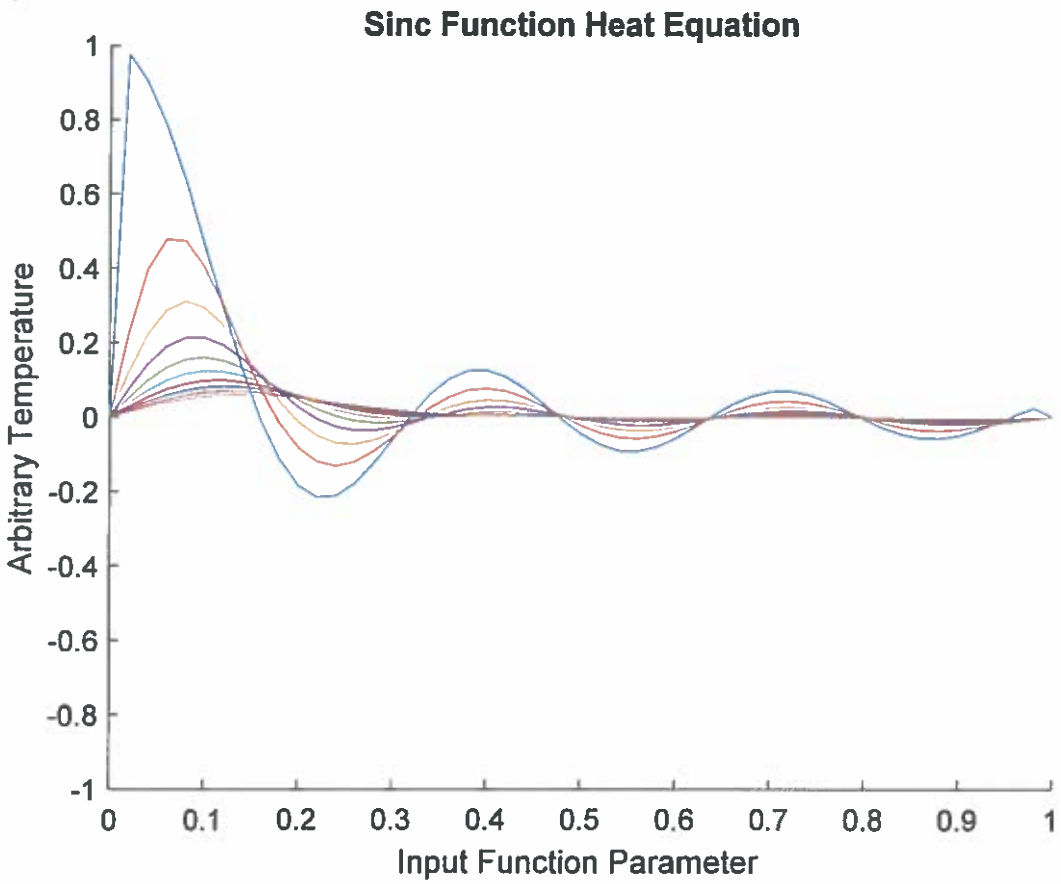


Figure 50

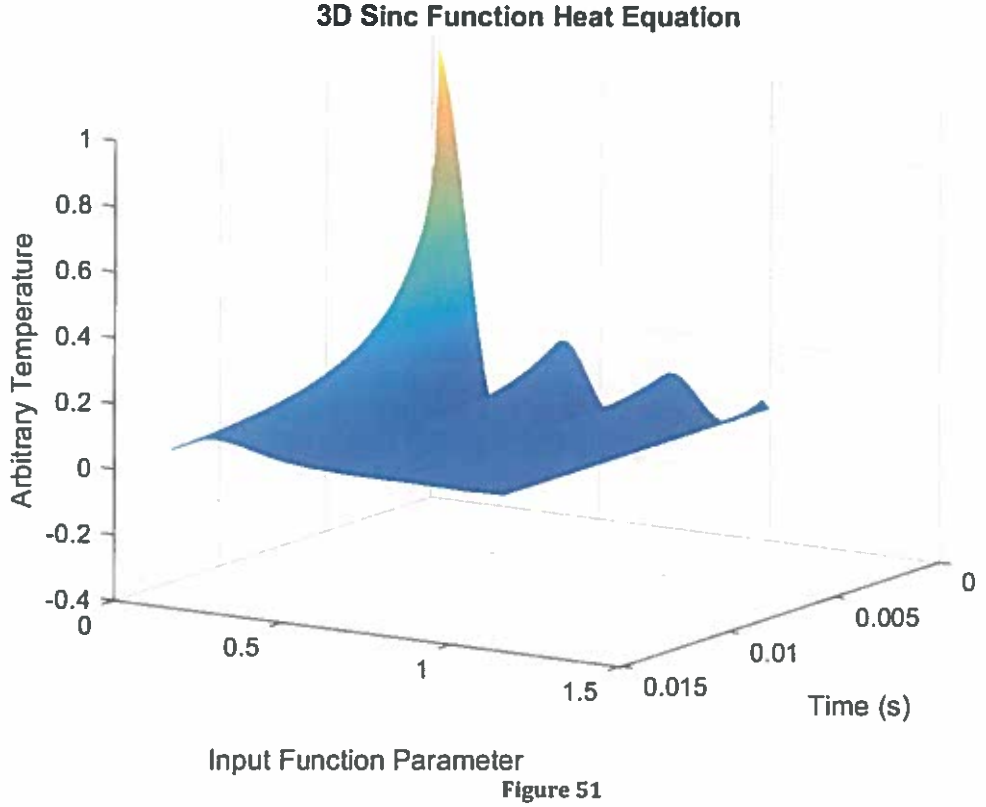


Figure 51

4.6 Sinc function constant boundary condition of 1

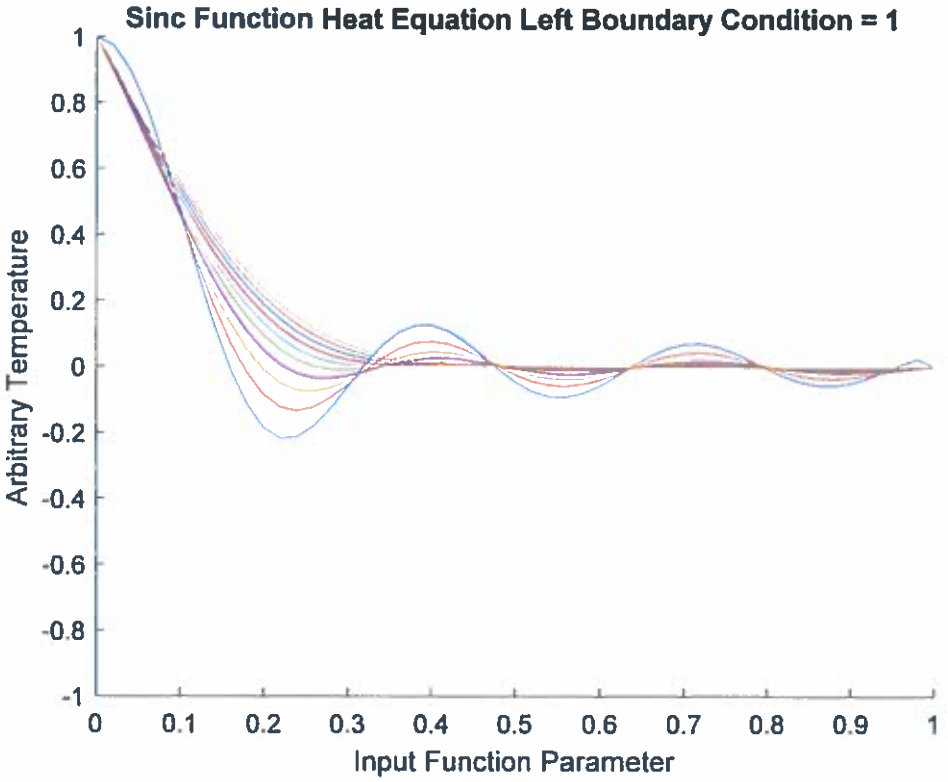


Figure 52

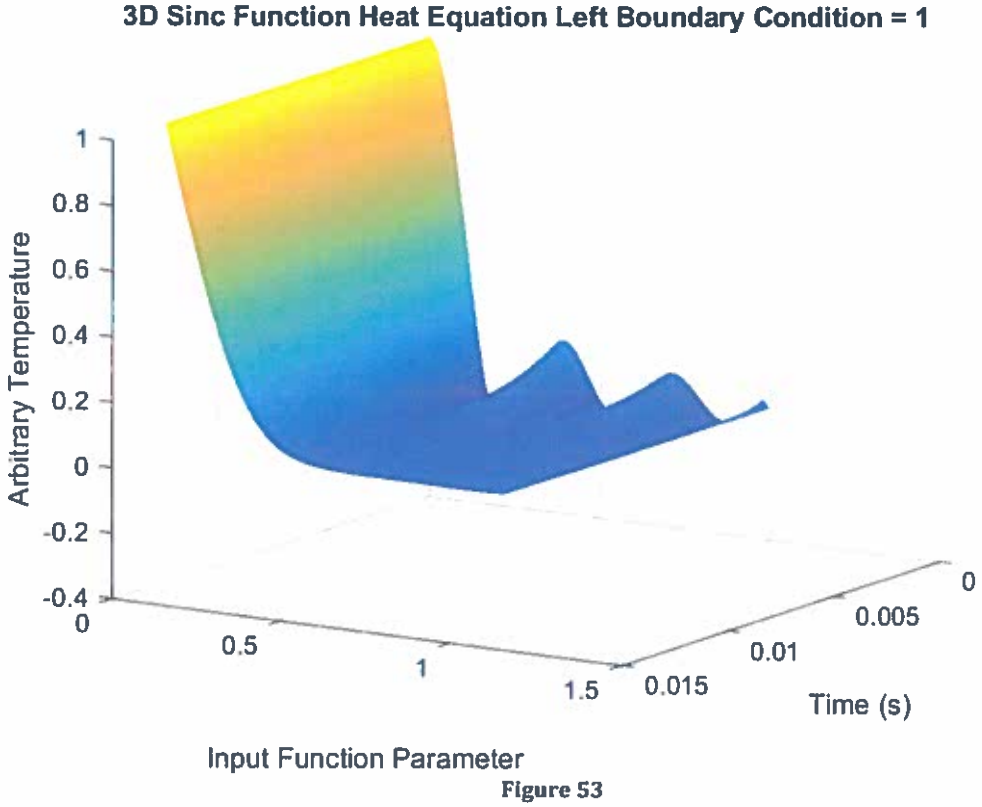


Figure 53

When the left boundary condition is set to 1 instead of zero, there are no instantaneous jumps at the left end of the rod, since *sinc* at 0 is 1. This lack of discontinuities is visible in the 2D plot, figure 52. However, as shown in figure 53, the effect of the main peak is much harder to see, since the effect of the left boundary condition constantly putting energy into the rod saturates the effect that would be visible from the initial condition

4.6 Sinc function varying boundary condition

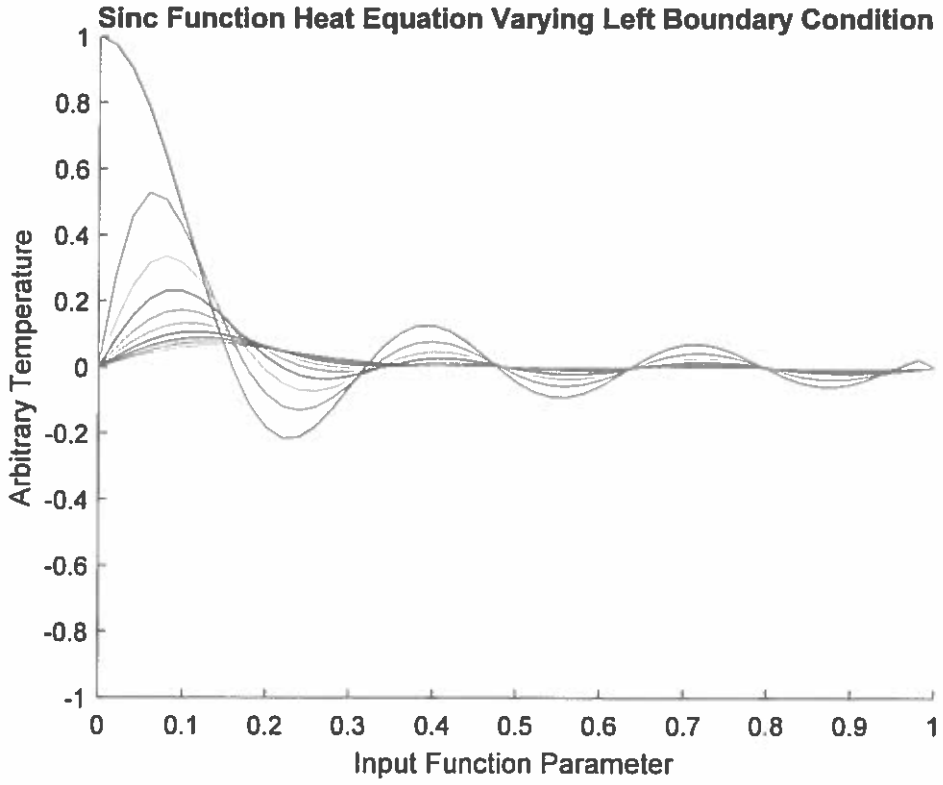


Figure 54

If it is known how the boundary should vary, then the 3D plot, figure 55, illustrates the benefit of using varying boundary conditions. In this case, the boundary condition has been chosen as a decaying exponential, which is similar to a non-numerical solution of the heat dissipation equation. This allows the numerical model to illustrate the heat dissipated without either completely distorting the dissipation over time by having a non-zero boundary condition that matches the initial condition, or having to force the initial conditions to change instantaneously at the boundary conditions.

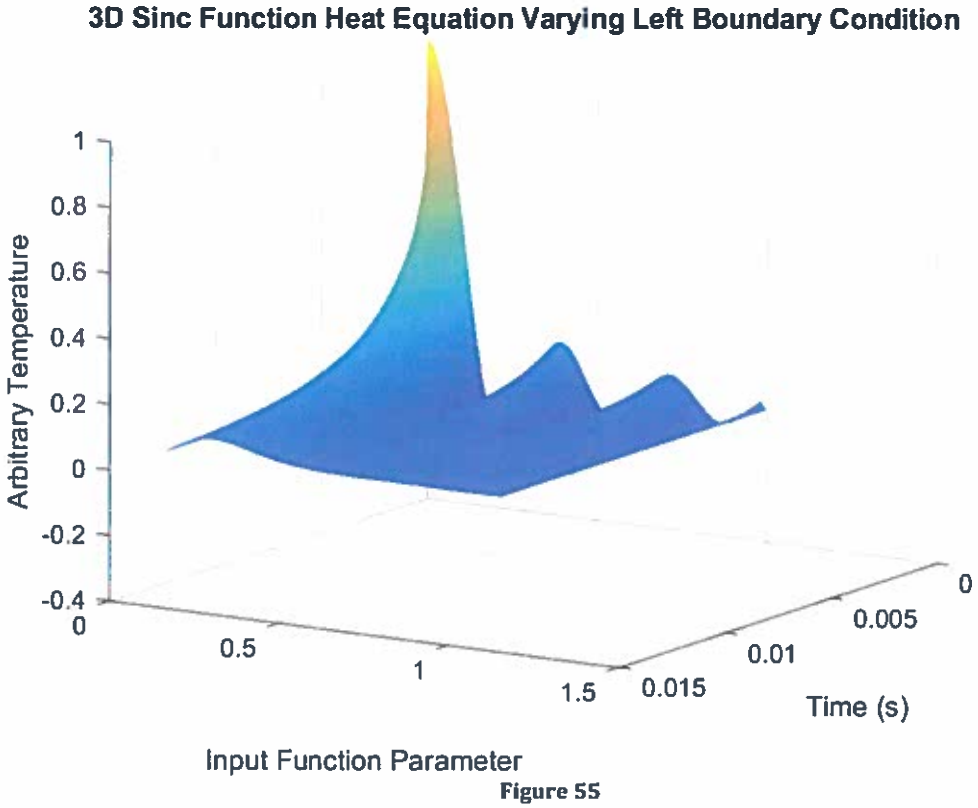


Figure 55

4.7 Hyperbolic tan function zero boundary condition

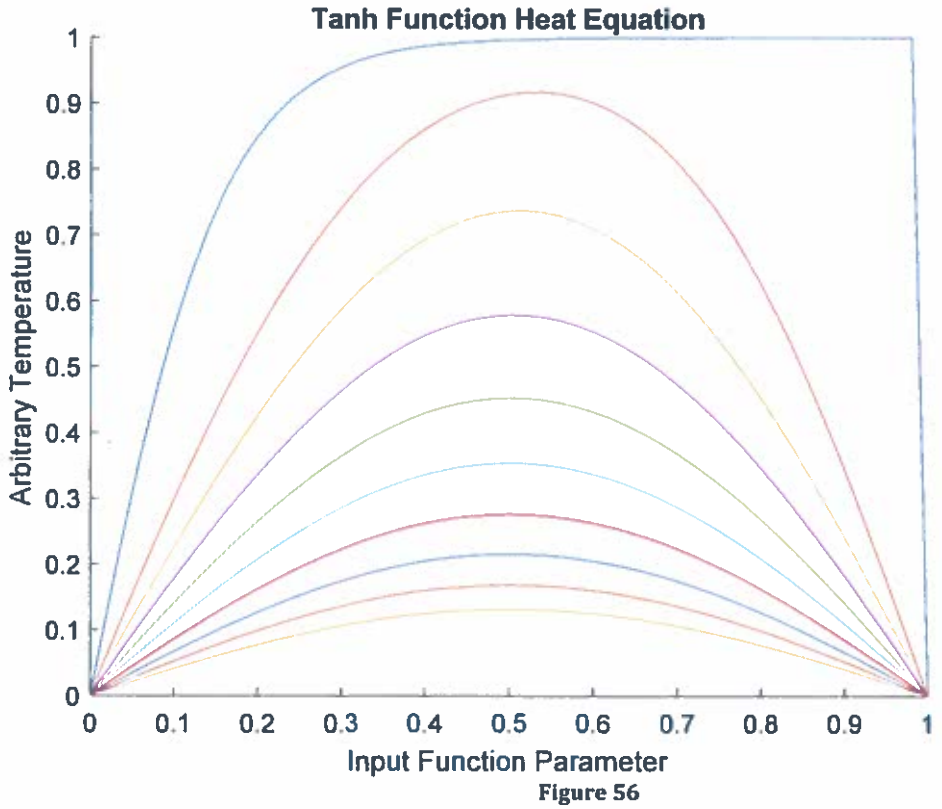


Figure 56

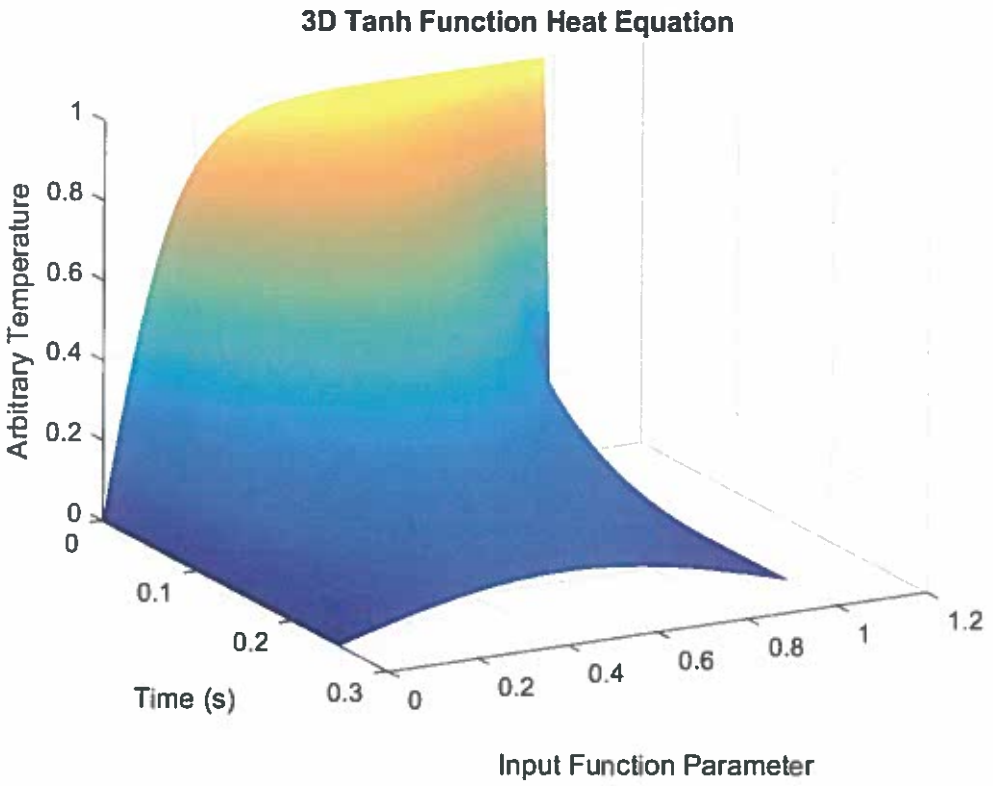


Figure 57

For hyperbolic tangent as an input condition, the distortion caused by a non-zero initial condition combined with a zero-boundary condition is more obvious. In the 3D plot, figure 57, the right portion of the plot is clearly dissipating faster than the center portion of the plot, despite the initial condition defining the rightmost part of the rod to have more heat energy than the middle portion.

4.7 Hyperbolic tan function constant right boundary condition

Again, setting the right boundary condition to match the value specified by the initial condition resolves the distortion. However, whilst matching the initial condition and boundary conditions removes any discontinuities, such as the discontinuity evident in figure 56, at $Input\ Function\ Parameter = 1$, and now not present in figure 58, it is now impossible to observe the behaviour of the system in an environment without a constant heat source being supplied to the right end of the rod. Again, a varying boundary condition is required to model the system in a state without a constant heat input. Both figures, 58 and 59 illustrate the equilibrium reached with a left boundary condition of 0 and right boundary condition of 1 is a linear temperature distribution through the rod.

good

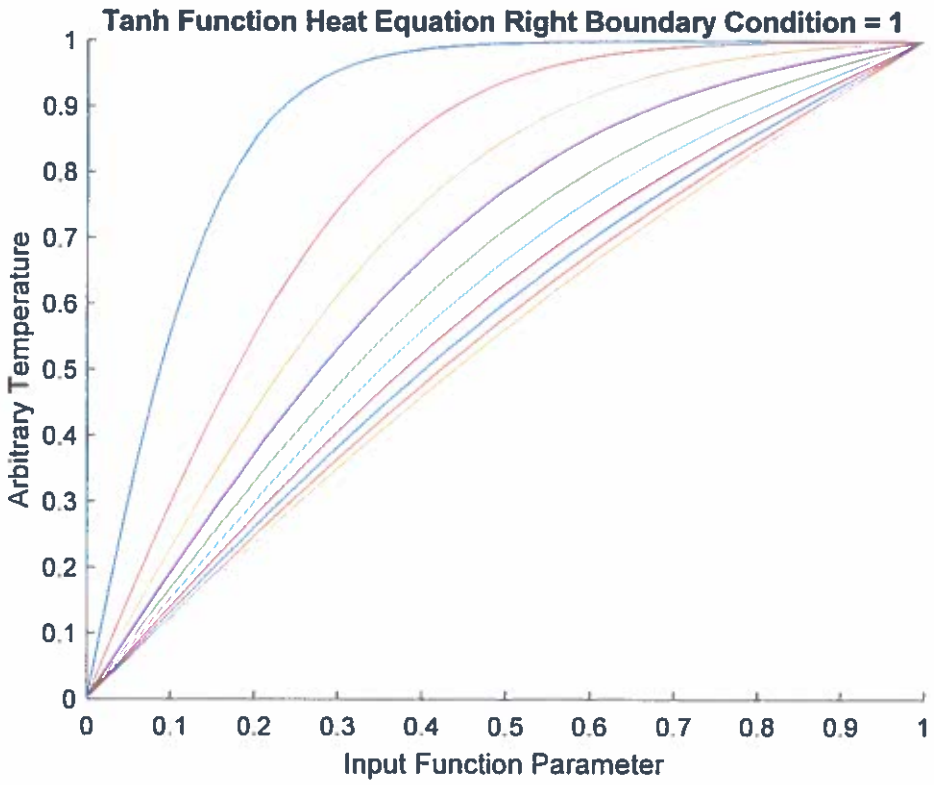


Figure 58

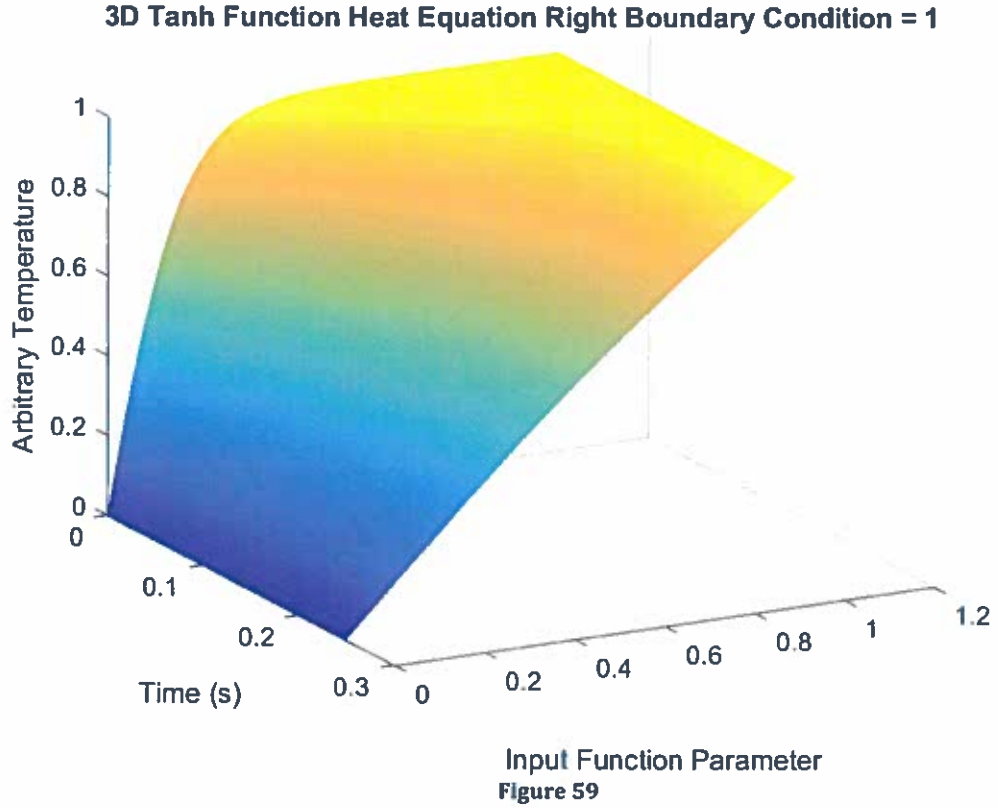


Figure 59

Appendix

Ex 1 Code

Table of Contents

Heuns Method 1
Midpoint Method 1
Ralston's Method 2
Input plot 3
Output plot 3
Overall Script 3
Inputs 3
Step Function 3
Impulsive Decay 4
Time to frequency 4

Heuns Method

```
function [times,Vout]= heuns(Vin,R,L,i0,h,N)

%set our inital conditions
i = i0;t = 0;
times(1) = 0;
Vout(1) = Vin(t) - i*R;

for j=2:N % loop for N steps
    t=t+h; % adding step size
    iPredict= i + h*(1/L)*(Vin(t)-R*i);% predictor for next y (Euler)

    % Average of previous and predicted(Heuns)
    i = i + h* ((  (0.5)*(  (Vin(t)-R*i) + (Vin(t+h)-R*iPredict) ) )/
 L);

    Vout (j) = Vin(t) - i*R; %Output voltage

    times (j) = t; %filling our time vector
end
```

Midpoint Method

```
function[t, vout] = midpoint(Vin,R,L,L_a,h,N)

% Setting the initial values to x_a
x_a = 0;

% Setting the initial values to y_a
y_a = L_a;

% Building and Initialising an array for t
t = zeros(1,N);
```

```
% Building and Initialising an array for iL
iL = zeros(1,N);

% Building and Initialising an array for Vout
vout = zeros(1,N);

% Setting the value of x_a to t
t(1)=x_a;

% Setting the value of y_a to iL
iL(1)=y_a;


% Taking loops as j-1 gradually increased to N-1
for j=1:N-1

    % The current values of t is taken as xt
    xt = t(j);

    % The current values of iL is taken as yt
    yt = iL(j);

    func = @(x,y) (Vin(xt) - R * yt) / L;

   % The middle value of the integral was taken and this will
 determine the slope of the graph for its local integral
    fx = xt + h / 2;
    fy = yt + h / 2 * feval(func, xt, yt);


   % As the iteration continuous, the current values of xt and yt were
 passed to the next iteration
    iL(j+1) = yt + h * feval(func, fx, fy);
    t(j+1) = xt+h;


   % The value of Vout was calculated base on the value of Vin and t
    vout(j) = Vin(xt) - R * yt;

end
```

Ralston's Method

```
function [times,Vout]= ralstons(Vin,R,L,i0,h,N)

%set our inital conditions
i = i0;t = 0;
times(1) = 0;
Vout(1) = Vin(t) - i*R;

%Ralstons: y(i+1) = y(i)+ h((k1)(1/3) + (k2)(2/3))
% where:
% k1 = f(xi,yi)
```

```
% k2 = f(xi + (3/4)h, yi + (3/4)(k1)h)

for j=2:N % loop for N steps
    t=t+h; % adding step size
    k1 =  (Vin(t) - i*R);
    k2= (Vin(t + (3*h)/4 ) - (i+(3/4)*k1*h)*R);
    i = i + h*((1/3)*k1 + (2/3) * k2 )/L;

    Vout (j) = Vin(t) - i*R; %Output voltage

    times (j) = t; %filling our time vector
end
```

Input plot

```
function [  ] = InputPlot( Vin,startT, endT, Name )
%figure;
fplot(Vin, [ startT endT],'b'); %plot our function as a blue line
title(Name) %give it our name
xlabel('Time(s)');ylabel('Voltage(V)'); % label axis

end
```

Output plot

```
function [  ] = OutputPlot( time,Vout,name )
plot(time,Vout,'r'); %plot our function as a red line
title(name);xlabel('Time(s)');ylabel('Voltage(V)'); %label axis
end
```

Overall Script

```
%Each input section will be seperated by a header

% Here we run all our inputs against Heuns, Midpoint and Ralstons

close all %close all our plots
```

Inputs

```
i0 = 0;%Initial condition
i = i0;
R = 0.5; %Resistance
L = 0.0015; %inductance
tf = 0.05; % end time
h=0.000001; %step size
N=round((tf-0)/ h); %number of steps
```

Step Function

```
Vin = @Step; %input function
```

```
tf = 0.02; % end time
h=0.000001; %step size
N=round((tf-0)/ h); %number of steps
[time,Vout] = heuns(Vin,R,L,i0,h,N); %Running heuns
%time,Vout] = midpoint(Vin,R,L,i0,h,N); %running midpoint
%[time,Vout] = Ralstons(Vin,R,L,i0,h,N);% running ralstons

%Plots input up to time 0.002, with name 'Step signal'
InputPlot ( Vin, 0 , 0.02, 'Step signal' );

%Plots Output with name 'Step Output'

OutputPlot (time ,Vout, 'Step Output');

% Impulsive Signal
tf = 0.05; % end time
h=0.000001; %step size
N=round((tf-0)/ h); %number of steps
Vin = @(t) 3.5*exp(-(t^2)/160e-6); %Input Impulsive signal

[time,Vout] = heuns(Vin,R,L,i0,h,N);
%[time,Vout] = midpoint(Vin,R,L,i0,h,N);
%[time,Vout] = Ralstons(Vin,R,L,i0,h,N);

figure;
InputPlot ( Vin, 0 , 0.05, 'Impulsive signal' );

hold on;
OutputPlot (time ,Vout,'Impulsive signal Output');
```

Impulsive Decay

```
tf = 0.005; % end time
N=round((tf-0)/ h); %number of steps

%Vin = @(t) 3.5*exp(-(t)/160e-6); %Input impulsive decay
Vin = @(t) 3.5*exp(-(t)/160e-6); %Input impulsive decay
%[time,Vout] = heuns(Vin,R,L,i0,h,N);
%[time,Vout] = midpoint(Vin,R,L,i0,h,N);
[time,Vout] = ralstons(Vin,R,L,i0,h,N);
figure;
InputPlot ( Vin, 0 , 0.005, 'Ralstons Impulsive Decay signal' );
hold on;
OutputPlot (time ,Vout,'Ralstons Impulsive Decay Output');
```

Time to frequency

We changed our time to frequency for easier comparison and input

```
% T = 20 e-6 => 50,000 Hz
% T = 160 e-6 => 6,250 Hz
% T = 450 e-6 => 20,000/9 Hz
% T = 1000 e -6 => 1000 Hz
```

```
%Here we have our frequencies with a correponding endtime to show
%a clear graph

Frequency = [ 1000,6250 ,(20000/9), 50000];

EndTime = [ 0.002, 0.0004 , 0.0009 , 0.00004]; %corresponding end
 times

% Sine/Sawtooth/Square Functions
sub = 0;

for i = 1 : 4
    sub = sub + 1;
    f = Frequency (i);    %get each frequency from our frequency array

    %change inputs
    %Vin = @(t) 4.5*square(2*pi*f*t);%SQUARE input
   %Vin = @(t) 4.5*sin(2*pi*f*t); %SINE input
    Vin = @(t) 4.5*sawtooth(2*pi*f*t);% SAWTOOTH input


    %Here we convert each of our frequencies to string for our axis
    name = num2str(f) ;
    Hz = ' Hz';
    name = strcat(name,Hz); %combine name and hz (name Hz)
    SineName = 'Heuns Saw Tooth Wave'; %Sine/Sawtooth/square
    OutputName = ' Output';
    Combined = [SineName, ' ' , name ];
    CombinedOut =  [Combined, OutputName]; %concatenate strings


    tf = EndTime(i); % end time (from array)
    N=round((tf-0)/ h); %number of steps



   [time,Vout] = heuns(Vin,R,L,i0,h,N); %run against heuns,mipoint or
 ralston
    %[time,Vout] = midpoint(Vin,R,L,i0,h,N);
    %[time,Vout] = ralstons(Vin,R,L,i0,h,N);

    subplot (2,2,sub); %subplot onto same graph
    InputPlot ( Vin, 0 , tf ,Combined); %plot input
    hold on ; % hold so that we plot onto same graph

    OutputPlot (time ,Vout,CombinedOut); %plot output

end
```

Published with MATLAB® R2016b

Exercise 2 Code

Table of Contents

Inputs 1
Solve ODE 1
Plot/Run 1
Ex2 log log graph 3
Inputs 3
Solve/Plot Exact ODE 3
Plot our loglog graph 4

Inputs

```
clear
close all
i0 = 0;%Initial condition
i = i0;
R = 0.5; %Resistance
L = 0.0015; %inductance
tf = 0.1; % end time
h=0.000001; %step size
N=round((tf-0)/ h); %number of steps

Vin = @(t) (6*cos(2*pi*(1/150e-6)*t)); % input cosine wave 150 us
```

Solve ODE

```
syms y(t) %y is our current

%Our differential eqn
eqn = diff (y,t) == (6*cos(2*pi*(1/150e-6)*t) - R* y)/L;

condition = y(0) == 0;% initial condition
sol = dsolve (eqn,condition); % Solve ODE
disp(sol); % display our solved solution



EsolF = matlabFunction(sol); %change our ODE solution to a function
z = 0:0.01/50 : 0.001; % For plotting
figure;
ezplot(sol,[z(1) z(end) ]); %Use ezplot to plot our exact solution
title('Exact solution');
```

Plot/Run

We are running heuns here, but to run midpoint/ralstons we simply change the function name

```
Hstepsize = [1e-4,1e-5,1e-6,1e-7,1e-8]; %Step size we are testing
```

```
sub =0; % for subplot
for k = 1:5 %5 step sizes
    sub = sub + 1; %increment every loop

    name = num2str(Hstepsize(k)) ; % set name = stepsize

    N=round((tf)/ Hstepsize(k)); %number of steps

    [time,Vout] = ralstonsI(Vin,R,L,i0,Hstepsize(k),N); % run heuns
    % [time,Vout] = midpointI(Vin,R,L,i0,Hstepsize(k),N); % run mid
 point
    % [time,Vout] = ralstonsI(Vin,R,L,i0,Hstepsize(k),N); % run
 ralstons

    for j = 1 : N
        Err (j) = EsolF(j) - Vout(j); %Calculate error

        worstErr(k) = max ( abs (Err(j) ) ); % this is our worst error

    end
    figure;
    plot (time,Vout);
    title('ralston');


    subplot (4,3,sub); %crease 4x3 subplot

    plot ( time , Err); % plot our error against time
    title(name);xlabel('Time(s)');ylabel('Error');

    %Here we adjust the axis to ensure we get a proper graph
    if (strcmp(name,'1e-06'))
        axis([ 0 0.2e-4 -0.2 0.2]);
    end
    if (strcmp(name,'1e-07'))
        axis([ 0 0.1e-4 -0.2 0.2]);
    end
    if (strcmp(name,'1e-08'))
        axis([ 0 0.8e-6 -0.2 0.2]);
    end

    %Ouput worst error
    fprintf ('%s worst error : %d\n ' , name , worstErr(k));
    %clear inputs
    clear time;
    clear name;
    clear Err;
end
```

Ex2 log log graph

Inputs

```
clear
close all
i0 = 0;%Initial condition
i = i0;
R = 0.5; %Resistance
L = 0.0015; %inductance
time = 0;
tf = 0.005; % end time
Vin = @(t) (6*cos(2*pi*(1/150e-6)*t)); % input cosine wave 150 us
```

Solve/Plot Exact ODE

```
syms i(t)
eqn = diff (i,t) == (6*cos(2*pi*(1/150e-6)*t) - R* i)/L; % ODE
 equation
condition = i(0) == 0;% initial condition
sol = dsolve (eqn,condition); % Solve ODE
disp(sol); % display our solved solution

EsolF = matlabFunction(sol); %change our ODE solution to a function
z = 0:0.01/50 : 0.001; % For plotting
figure;
ezplot(sol,[z(1) z(end) ]); %Use ezplot to plot our exact solution
title('Exact solution');

figure;
N= 16;
for(k=3:N) % step sizes

    h = 2^(-k);
    %N=round((tf-0)/ h); %number of steps
    [time,VoutH] = heuns(Vin,R,L,i0,h,N); % run heuns
    [timeM,VoutM] = midpoint(Vin,R,L,i0,h,N); % run mid point
    [timeR,VoutR] = ralstons(Vin,R,L,i0,h,N); % run ralstons

    ExactH = (6*cos(2*pi*(1/150e-6)*time));
    ExactM = (6*cos(2*pi*(1/150e-6)*timeM));
    ExactR = (6*cos(2*pi*(1/150e-6)*timeR));
    ErrH = ExactH - VoutH; %Calculate error
    ErrM = ExactM - VoutM;
    ErrR = ExactR - VoutR;

    worstErrH = max ( abs (ErrH)); % this is our worst error
    worstErrM = max ( abs (ErrM)); % this is our worst error
    worstErrR = max ( abs (ErrR)); % this is our worst error

    %Ouput worst error
    fprintf ('Heuns: %s worst error : %d\n ' , h , worstErrH);
```

```
    fprintf ('Ralstons: %s worst error : %d\n ' , h , worstErrR);
    fprintf ('Midpoint: %s worst error : %d\n ' , h , worstErrM);
```

Plot our loglog graph

```
    plot ( log(h),log(worstErrH), 'r*'); %log log graph
    hold on;
    plot (log(h), log(worstErrM), 'b*');
    hold on;
    plot (log(h), log(worstErrR), 'g*');
    hold on;
    title('Log Log
Methods');xlabel('log(Stepsize)');ylabel('log(Error)'); %labels

legend('Heuns', 'Midpoint', 'Ralstons', 'Location', 'northwest') %legends

end
```

Published with MATLAB® R2016a

Ex 3 Code

Table of Contents

RK4 Second 1
RLC Coupled 2
Run 2
Step 3

RK4 Second

```
function [Xi,Yi] = RK4second(x, y, h, rlc_f, v)
% RK4 3/8 method for a single time step
% Parmeters:
% x - time
% y - output
% h - time step
% rlc_f - RLC coupled first order ODEs function
% v - Input function v(x), returns voltage given a time

  % This is a direct implementation of the Runge-Kutta
  % 4th order 3/8 method. It's called the 3/8 method
  % due to the weighting applied when summing the values
  % of k1, k2, k3 and k4

  % Calculate k1 by applying the coupled equation to the
  % current input and output voltage
  % then mutliply each element by the stepsize
  k1 = rlc_f(v(x), y).* h;

  % Calculate k2 by applying the coupled equation to the
  % input voltage in x+h/3 seconds,
  % and output y - k1/3
  % then multiply each element by the stepsize
  k2 = rlc_f(v(x+(h/3)), y + (k1./3)) .* h;

  % Calculate k3 by applying the coupled equation to the
  % input voltage in x+(2*h/3) seconds,
  % and output y - k1/3 + k2
  % then multiply each element by the stepsize
  k3 = rlc_f(v(x+2*(h/3)), (y- (k1./3))+k2) .* h;

  % Calculate k4 by applying the coupled equation to the
  % input voltage in x+h seconds,
  % and output y + k1 + k3 - k2
  % then multiply each element by the stepsize
  k4 = rlc_f(v(x+h), ((y+k1)+k3)-k2) .* h;

  % The final value of y is a weighted sum of each of the
```

```
    % k terms, divided by 8. This weighting is the source of
    % the name for this method
    Yi = y + ((k1 + k2.*3 + k3.*3 + k4) ./ 8);

    % Increment time by the step size
    Xi = x + h;

end
```

RLC Coupled

```
function [yn] = RLC_coupled(x,y,R,L,C)
% This function sets up the two coupled
% first order ODEs in q and q'

% Input parameters:
% x - time
% y - y(1) = q, y(2) = q'
% Output parameters:
% yn(1) = q, yn(2) = q': next values of q and q'

    % Set default values of RLC as described in the project brief
    % if they are not specified by the function
    % R is 280 Ohms
    if ~exist('R', 'var') R = 280; end
    % L is 600e-3 H
    if ~exist('L', 'var') L = 600e-3; end
    % C is 4e-6 F
    if ~exist('C', 'var') C = 4e-6; end

    % Create the yn array.
    yn = zeros(1);

    % Use coupled equations: separate the second order equation into
    % two coupled first order equations to calculate next values of q
 and q'

    % q = q', the first coupled equation
    yn(1) = y(2);
    % This is the second coupled equation
    % q' = (x- R*q' - q/C)/L
    yn(2) = (x - R*y(2) - y(1)/C)/L;
end
```

Run

```
function [ time, vin, vout ] = run(q, q_, h, x, x_end, in, eq)
% Runs the Runge-Kutta method across the time interval specified,
 using h as step size.
% Input parameters:
% q - q initial (see brief for further information on q and q')
% q - q' initial
% h - time step size
```

```
% x - start time, usually 0
% x_end - end time
% in - function handle for input voltage function
% eq - function handle for coupled first order ODEs equation function
% Output parameters:
% time - vector of time values, incremented by h each time
% vin  - vector of input voltage values
% vout - vector of output voltage values

    % Put initial conditions into input vector
    y(1) = q; % q = 500nF
    y(2) = q_; % q' = 0

    % Calculate the number of steps from the start/finish times and
 step size
    num_steps = (x_end-x)/h;

    % Iterate Runge-Kutta for the number of steps
    for i = 1:num_steps
        % vout is equal to q' * R, where R is 280 Ohms
        vout(i) = y(2)*280;
        % vin in is equal to input_f(x)
        vin(i) = in(x);
        % time = x
        time(i) = x;
        % Get next values of x and y from RK4second
        % x will be x+h
        % y will contain the next values of q and q'
        % calculated using the Runge-Kutta 3/8 method
        [x,y] = RK4second(x, y, h, eq, in);

    end
end
```

Step

```
function [ v ] = step(t)
% A function which acts as a step function
% that equals zero until t>0
% if t>0, v is a constant 5
% Input parameters:
% t - input time
% Output parameters:
% v - output voltage
  if t>0
      v = 5;
  else
      v = 0;
  end
end
```

Published with MATLAB® R2016a

Ex3 Script and Response plot

Table of Contents

RLC Script 1
Input response plot 4

RLC Script

RLC_script.m Runs the Runge-Kutta 3/8 methods for varying system parameters and input functions and plots the results

```
% Initial conditions
q = 500E-9; % q = 500nF
q_ = 0; % q' = 0

% Create a function handle to couple first order ODEs
% to pass to Runge-Kutta method
eq = @RLC_coupled;

% Start at t=0 always
start_time = 0;

% RUN AND PLOT FOR STEP FUNCTION

h = 10e-5;      % Step size
end_time = .05; % End time 0.05s to capture interesting part of the
 plot

% Create function handle for input voltage function using step
 function
input = @step;

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, 'Step Function');

% Print the graph to pdf
print('screenshots/step.pdf','-dpdf');

% END OF STEP FUNCTION RESULTS

% RUN AND PLOT FOR DECAYING STEP FUNCTION

h = 10e-5;      % Step size
end_time = .05; % End time 0.05s to capture interesting part of the
 plot
```

```
% Create a function handle for a decaying step function with amplitude
 5 and T = 3e-6
input = @(t)5*exp(-(t^2)/3e-6);

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, 'Decaying step');

% Print the graph to pdf
print('screenshots/decaying_step.pdf','-dpdf');

% END OF DECAYING STEP FUNCTION RESULTS

% RUN AND PLOT FOR 5Hz sinewave

h = 10e-5;      % Step size
end_time = 0.3; % End time 0.3s to capture interesting part of the
 plot
% Create a function handle for a 5Hz sinewave with amplitude 5
input = @(t)(5*sin(2*pi*5*t));

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, '5Hz Sine wave');

% Print the graph to pdf
print('screenshots/5Hz_sine.pdf','-dpdf');

% END OF 5 Hz sinewave

% RUN AND PLOT FOR 110Hz sinewave

h = 10e-5;       % Step size
end_time = 0.05; % End time 0.05s to capture interesting part of the
 plot
% Create a function handle for a 110Hz sinewave with amplitude 5
input = @(t)(5*sin(2*pi*110*t));

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, '110Hz Sine wave');

% Print the graph to pdf
print('screenshots/110Hz_sine.pdf','-dpdf');
```

```
% END OF RESULTS FOR 110Hz sinwave

% RUN AND PLOT FOR 500Hz sinewave
start_time = 0;
end_time = 0.02;
h = 10e-6; % Step size
% Create function handle for a 500Hz sine wave with amplitude 5
input = @(t)(5*sin(2*pi*500*t));

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, '500Hz Sine wave');

% Print the graph to pdf
print('screenshots/500Hz_sine.pdf','-dpdf');

% END OF RESULTS FOR 500Hz sinewave

% RUN AND PLOT FOR 5Hz squarewave

start_time = 0;
end_time = 0.3;
h = 10e-5; % Step size
% Create function handle for a 5Hz square wave
input = @(t)(5*square(2*pi*5*t));

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, '5Hz Square wave');

% Print the graph to pdf
print('screenshots/5Hz_square.pdf','-dpdf');

% END OF RESULTS FOR 5Hz squarewave

% RUN AND PLOT FOR 110  Hz squarewave

start_time = 0;
end_time = 0.05;

% Create function handle for a 110Hz square wave
input = @(t)(5*square(2*pi*110*t));

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
```

```
input_response_plot(time, vin, vout, '110Hz Square wave');

% Print the graph to pdf
print('screenshots/110Hz_square.pdf','-dpdf');

% END OF RESULTS FOR 110Hz squarewave

% RUN AND PLOT FOR 500Hz squarewave

start_time = 0;
end_time = 0.02; % End time set to 0.02 to capture interesting
 behaviour
h = 10e-6; % Step size
% Create function handle for a 500Hz square wave
input = @(t)(5*square(2*pi*500*t));

% Run the Runge-Kutta 3/8 method across the specified time period
% Store results in vectors time, vin and vout
[time,vin,vout] = run(q, q_, h, start_time, end_time, input, eq);

% Plot the results
input_response_plot(time, vin, vout, '500Hz Square wave');

% Print the graph to pdf
print('screenshots/500Hz_square.pdf','-dpdf');

% END OF RESULTS FOR 500HZ squarewave
```

Input response plot

```
function input_response_plot(time,vin,vout,t)
% Plots input voltage and corresponding
% output voltage against time in the same figure

% Input Parameters:
% time - array of time values, incremented by the time step
% vin  - array of input voltages, from evaluating f(x) at each time
 step
% vout - array of output voltages, from evaluating the coupled first
 order
%        ODEs using the Runge-Kutta 3/8 method
% t    - title of the plot
    % Create the window for the plot
    figure;
    % Create a subplot at the top of the figure for input voltage plot
    subplot(2,1,1);
    % Plot time against vin in red
    plot(time, vin, '-r');
    % Add the title and axis labels for top plot
    title(t); xlabel('Time (s)'); ylabel('Amplitude (V)');
    % Create a subplot at the bottom of the figure for output voltage
 plot
    subplot(2,1,2);
```

```
    % Plot time against vout in blue
    plot(time, vout, '-b');
    % Add the title and axis labels for bottom plot
    title('Response'); xlabel('Time (s)'); ylabel('Amplitude (V)');
end
```

Published with MATLAB® R2016a

Ex 4 Code

Table of Contents

Finite Script 1
Finite Differences 7
Tent function 8

Finite Script

```
N = 50; % number of intervals
h = 1/N; % step size
k = 0.00005; % discretize time into intervals of k
t = N*100; % number of time intervals

% SOLVE FOR TENT FUNCTION

% Create a function handle for the tent function
func = @tent;

% Run finite difference function using tent function
% Store results in U and g
[U,g] = FiniteDifference(h,k,N,t,func,@(t)0,@(t)0);

% Plot for tent function
figure; % Create a figure
axis([0 1 0 1]); % Set the axis to the boundaries

hold on; % Plot all lines in the same plot
for m = 1:t
    if (mod(m,500) == 1) % Only plot every 500 time intervals
        plot(g, U(m,:)); % Plot time against a specific j row in U
    end
end
hold off; % stop plotting on same plot
title('Tent Function Heat Equation');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/tent2d.eps', '-depsc');

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D Tent Function Heat Equation');
zlabel('Arbitrary Temperature'); % Label z axis Arbitrary temperature.
 Doesn't have any units since it is arbitrary
```

```
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(122,15)
print('screenshots/tent3d.eps', '-depsc'); %

% SOLVE FOR SIN(2pix)
k = 0.000025; % reduce time step for sinusoidal input functions to
 plot finer details
% Create a function handle for sinewave with frequency 1Hz
func = @(t)(sin(2*pi*t));

% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(t)0,@(t)0);

% Plot for sine function
figure; % Create a figure
axis([0 1 -1 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1) % Only plot every 500 m
        plot(g, U(m,:));
    end
end
hold off;
title('1Hz Sinewave Heat Equation');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/sine2d.eps', '-depsc'); %

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D 1Hz Sinewave Heat Equation');
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(12,28)
print('screenshots/sine3d.eps', '-depsc'); %

% SOLVE FOR ABS SIN(2pix)

% Create a function handle for abs sinewave with frequency 1Hz
func = @(t)abs((sin(2*pi*t)));

% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(t)0,@(t)0);
```

```
% Plot for abs(sine) function
figure; % Create a figure
axis([0 1 -1 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1)
        plot(g, U(m,:));
    end
end
hold off;
title('1Hz Abs Sinewave Heat Equation');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/absine2d.eps', '-depsc'); %

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D 1Hz Abs Sinewave Heat Equation');
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(12,28)
print('screenshots/absine3d.eps', '-depsc');%

% SOLVE FOR SINC FUNCTION
k = 0.0000025; % reduce time step for sinusoidal input functions to
 plot finer details
% Create a function handle for sinc function
func = @(t)(sinc(t*2*pi));
% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(t)0,@(t)0);

% Plot for sinc function
figure; % Create a figure
axis([0 1 -1 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1)
        plot(g, U(m,:));
    end
end
hold off;
title('Sinc Function Heat Equation');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/sinc2d.eps', '-depsc');

figure; % Make new figure
```

```
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D Sinc Function Heat Equation');
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(122,15)
print('screenshots/sinc3d.eps', '-depsc');

% SOLVE FOR SINC FUNCTION BOUNDARY CONDITION = CONSTANT

% Create a function handle for sinc function
func = @(t)(sinc(t*2*pi));
k = 0.0000025; % discretize time into intervals of k
% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(t)1,@(t)0);

% Plot for sinc function
figure; % Create a figure
axis([0 1 -1 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1)
        plot(g, U(m,:));
    end
end
hold off;
title('Sinc Function Heat Equation Left Boundary Condition = 1');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/sincconstnzbound2d.eps', '-depsc');

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D Sinc Function Heat Equation Left Boundary Condition = 1');
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(122,15)
print('screenshots/sincconstnzbound3d.eps', '-depsc');

% SOLVE FOR SINC FUNCTION BOUNDARY CONDITION VARYING
```

```
% Create a function handle for sinc function
func = @(t)(sinc(t*2*pi));
k = 0.0000025; % discretize time into intervals of k
% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(q)(exp(-((q-1)/100))),@(t)0);

% Plot for sinc function
figure; % Create a figure
axis([0 1 -1 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1)
        plot(g, U(m,:));
    end
end
hold off;
title('Sinc Function Heat Equation Varying Left Boundary Condition');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/sincvarybound2d.eps', '-depsc');

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D Sinc Function Heat Equation Varying Left Boundary
 Condition');
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(122,15)
print('screenshots/sincvarybound3d.eps', '-depsc');


% SOLVE FOR TANH BOUNDARY CONDITION != INITIAL CONDITION
k = 0.00005; % discretize time into intervals of k
t = N*100;
% Create a function handle for TANH
func = @(t)(tanh(2*pi*t));

% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(t)0,@(t)0);

% Plot for tanh function
figure; % Create a figure
axis([0 1 0 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1)
        plot(g, U(m,:));
```

```
    end
end
hold off;
title('Tanh Function Heat Equation');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/tanh2d.eps', '-depsc');

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D Tanh Function Heat Equation');
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(62,23)
print('screenshots/tanh3d.eps', '-depsc');

% SOLVE FOR TANH BOUNDARY CONDITION CONSTANT VALUE == INITIAL
 CONDITION
k = 0.00005; % discretize time into intervals of k
t = N*100;
% Create a function handle for TANH
func = @(t)(tanh(2*pi*t));

% Run finite difference function using function handle
[U,g] = FiniteDifference(h,k,N,t,func,@(t)0,@(t)1);

% Plot for tanh function
figure; % Create a figure
axis([0 1 0 1]) % set the axis to the boundaries
hold on;
for m = 1:t
    if (mod(m,500) == 1)
        plot(g, U(m,:));
    end
end
hold off;
title('Tanh Function Heat Equation Right Boundary Condition = 1');
xlabel('Input Function Parameter');
ylabel('Arbitrary Temperature');
print('screenshots/tanhconstantbound2d.eps', '-depsc');

figure; % Make new figure
xMesh = [1:t+1]; % Create array of same size as U rows and fill it
 with indexes
surf(xMesh.*k, g, U'); % 3D plot the heat diffusion result versus time
 and the time stamp of the input function
title('3D Tanh Function Heat Equation Right Boundary Condition = 1');
```

```
zlabel('Arbitrary Temperature'); % Label z axis Temperature. Doesn't
 have any units since it is arbitrary
xlabel('Time (s)');  % Label x axis time
ylabel('Input Function Parameter'); % Label y axis Input function
 time. Doesn't have any units since its arbitrary
shading interp
view(62,23)
print('screenshots/tanhconstantbound3d.eps', '-depsc');
```

Finite Differences

```
function [ U,g ] = FiniteDifference(h, k, N, t, func, b1, b2)
%FiniteDifference.m
% Perform finite difference method
% Input Parameters:
% h - step size,
% k - time interval
% N - number of steps,
% t - total diffusion time
% func - input function handle
% b1 - variable boundary condition 1
% b2 - variable boundary condition 2
% Output:
% U - U(mj) matrix holding m and j output values
% g - vector holding x-axis (time) values for plotting


v = k/(h^2);

% 2-D ouput vector created the size of the number of intervals
% containing m and j for the finite difference method
U = zeros(N,N); % U(m,j)
x = 0;



% Created a 1D vector of used for storing x-axis values
g = zeros(1,N); % used for the x-axis

% Define the first row according to the initial conditions
for a = 1:N+1
   % Store f(x) as U(j), u(M) is time intervals, and U(1,:) is the
 first row
   U(1, a) = func(x);
   % Store x-axis value in g, for use in the plot
   g(a) = x;
   % Increment x by the step size
   x = x + h;
end

% Set the boundary conditions, usually zero, but for extension non-
zero or
```

```
% variable. Choose to overwrite initial condition with boundary
 condition
% when they are not equal.
for m = 1:t+1
  U(m,1) = b1(m);
  U(m,N+1) = b2(m);
end

% For each of the time intervals
% For each of the columns in U except the last column
for m = 1:t
    % For each of the rows in U except for the boundary condition
 columns
    for a = 2:N
        % Applying finite difference method, this is the main
 algorithm.
        % We calculate the column below the current column using the
 finite differences
        % between three values in the current column. This is a direct
 implementation
        % of the finite different method. We can't caclulate the
 boudary values since
        % they would require values out of the bounds of the array, so
 these are defined
        % earlier on line 40.
        U(m+1, a) = v*U(m, a-1) + (1-(2*v))*U(m,a) + v*U(m,a+1);
    end
end
end
```

Tent function

```
function [ v ] = tent(x)
% tent.m
% Tent function around 0.5
    if x<=0.5
        v = 2*x;
    else
        v = 2 - 2*x;
    end
end
```

Published with MATLAB® R2016a